



Université de Rouen Normandie

SSL Checker

Cahier des Tests

Cahier des Tests

Étudiants :

Mehdi LASMI

Marzouk MONIR

Malik MAHFOUDI

Karina AIT-ABDELKAOUI

Naby TRAORE SAVANE

Rayan BOUGHANEM

Hamza RAHMOUNI

Amr ELGOHARY

Client :

Bruno MACADRÉ

Table des matières

1 Module Cipher Suite **2**

1.1 Test sur des sites qui supportent des protocoles obsolètes SSLv2 et SSLv3 . 9

2 Module Certificat **9**

2.1 Le cas parfait 9

2.2 Certificat expiré 11

2.3 Certificat auto-signé 11

2.4 Certificat révoqué 12

2.5 Signature faible (SHA-1 ou MD5) 13

2.6 Chaîne de certification incomplète 13

3 Le Front-end **14**

4 Module Protocole **19**

4.1 Test sur les protocoles TLS 20

4.2 Test sur les protocoles SSL 20

5 Module Compatibilité OS **22**

6 Module Échange de clé **26**

6.1 Test sur vpn2.gulfstream.ru 27

6.2 Test sur mail.anafra.cz 27

6.3 Test sur eminwon.haman.go.kr 28

7 Module Vulnérabilités **29**

7.1 Test sur www.google.com 29

7.2 Test sur : host.docker.internal Port : 4433 32

7.3 Test sur www.youtube.com 33

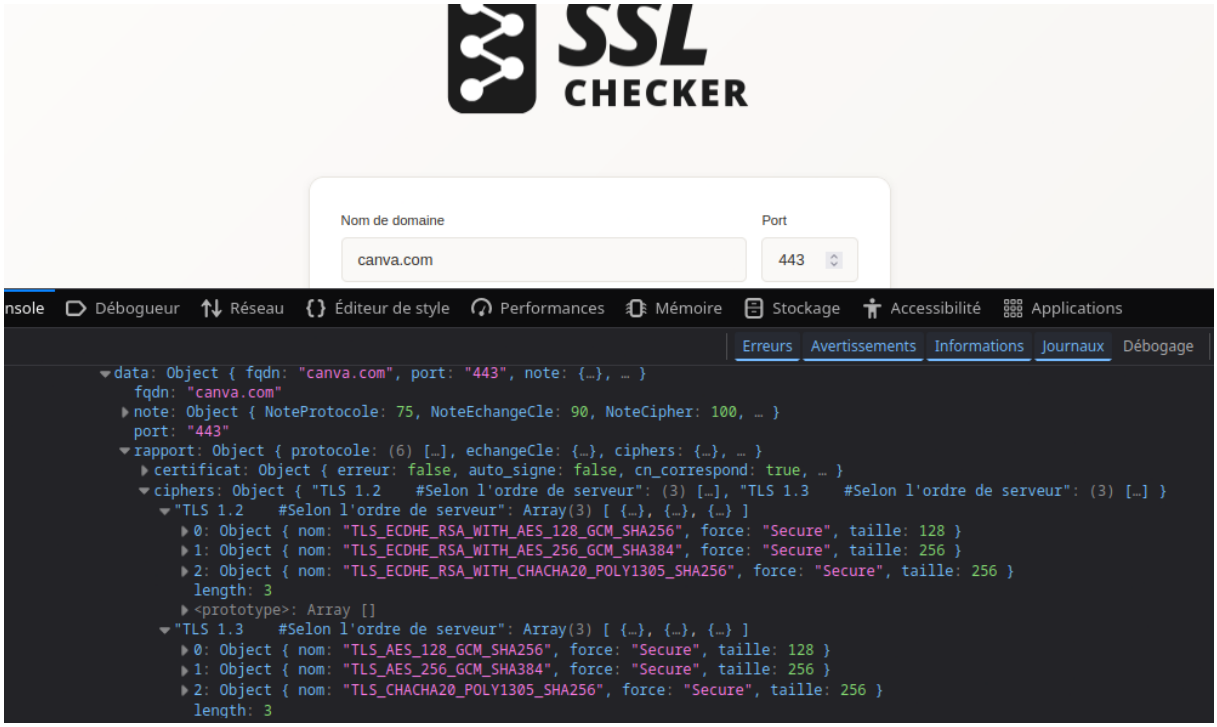
7.4 Vérification sur hstspreload.org 33

8 Module Scan de Ports **33**

8.1 Test rapide sur sogo.univ-rouen.fr 33

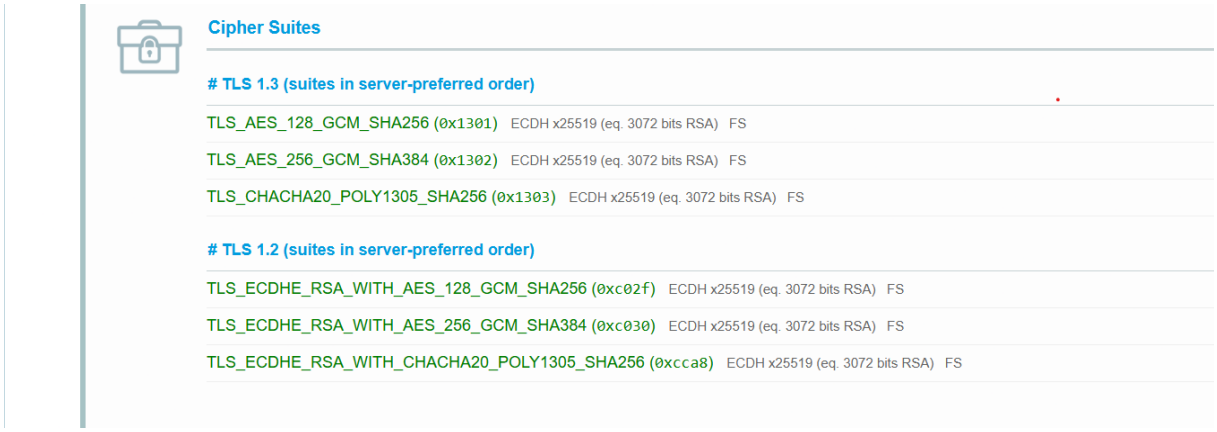
8.2 Test complet sur mdw.univ-rouen.fr 34

1 Module Cipher Suite

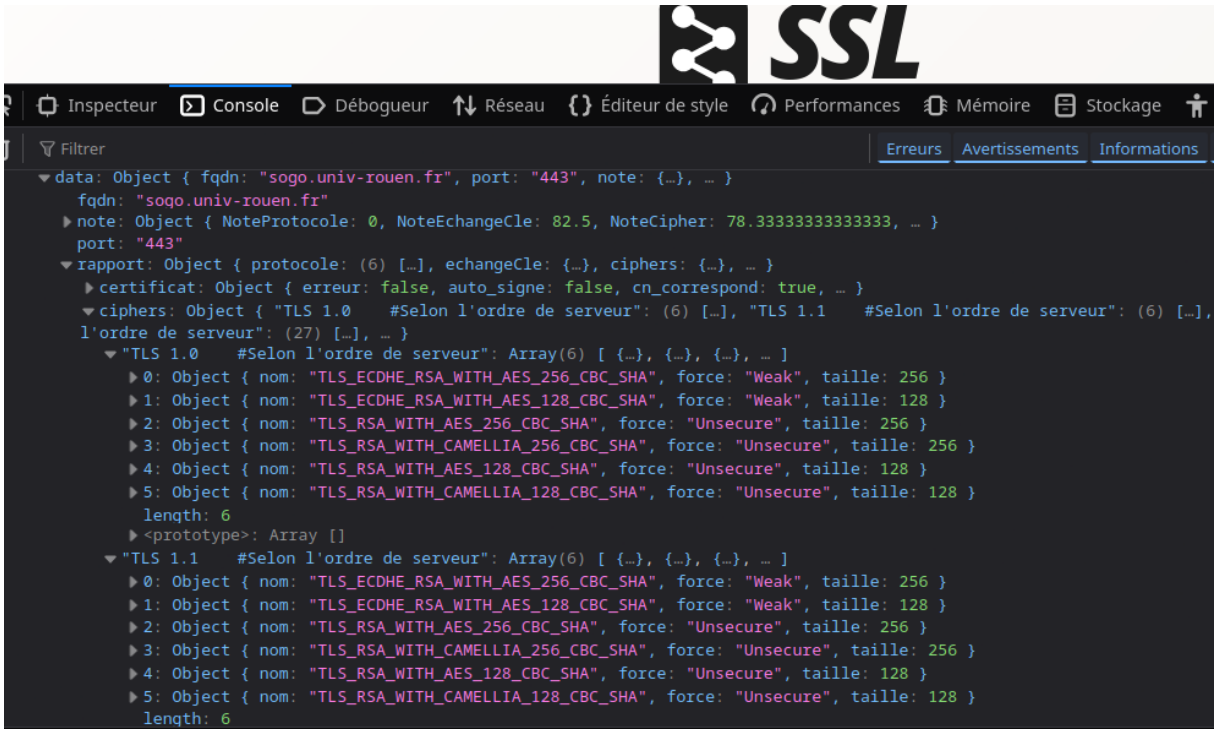


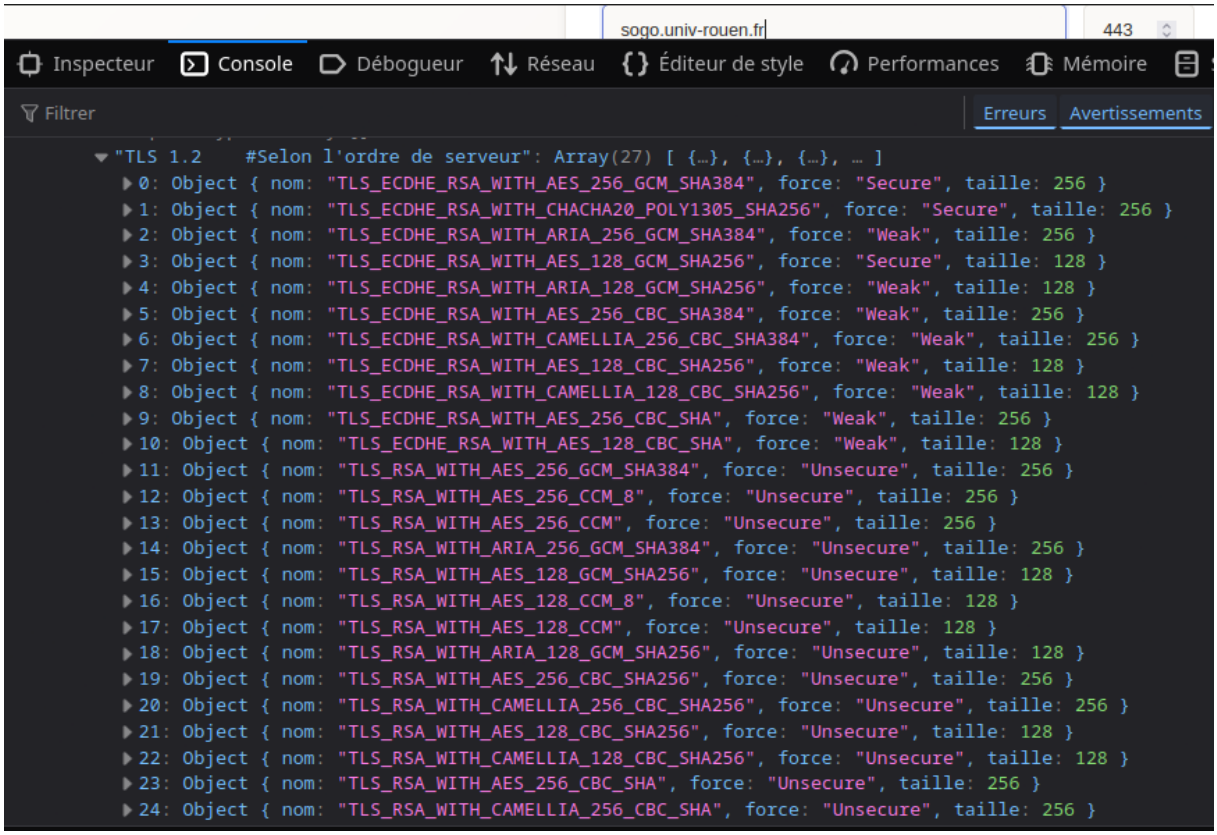
Nous avons éprouvé ce module sur des serveurs particulièrement exigeants quant aux configurations requises côté client, tels que le service de messagerie de l'université (sogo.univ-rouen.fr) ou la plateforme Canva (canva.com). À l'issue de nos tests (portant sur l'énumération et l'ordre de préférence des suites cryptographiques), nous obtenons des résultats en tout point identiques à ceux de Qualys SSL Labs, tout en garantissant des temps de réponse très courts.

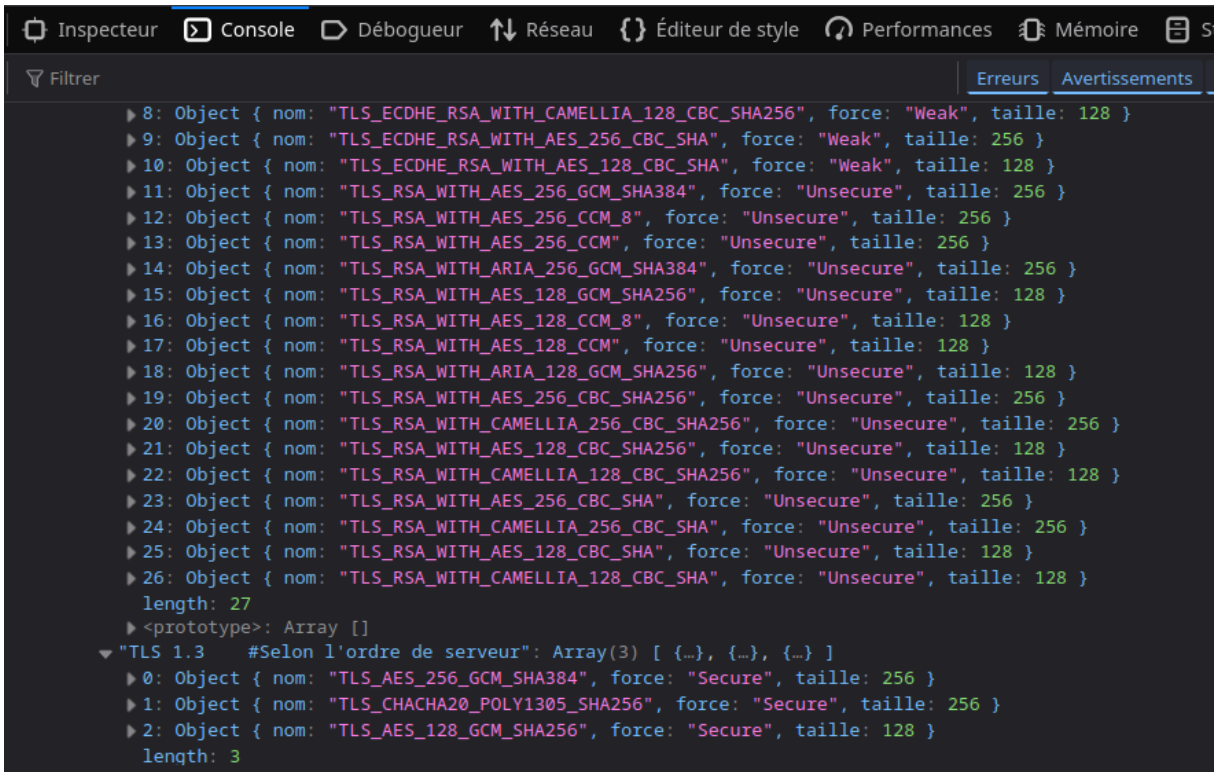
Test sur canva.com



Test sur sogo.univ-rouen.fr









Cipher Suites

TLS 1.3 (suites in server-preferred order)

TLS_AES_256_GCM_SHA384 (0x1302) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_CHACHA20_POLY1305_SHA256 (0x1303) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_AES_128_GCM_SHA256 (0x1301) ECDH x25519 (eq. 3072 bits RSA) FS

TLS 1.2 (suites in server-preferred order)

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca8) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_ARIA_256_GCM_SHA384 (0xc061) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_ARIA_128_GCM_SHA256 (0xc060) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 (0xc028) ECDH x25519 (eq. 3072 bits RSA) FS **WEAK**

TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384 (0xc077) ECDH x25519 (eq. 3072 bits RSA) FS **WEAK**

TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 (0xc027) ECDH x25519 (eq. 3072 bits RSA) FS **WEAK**

TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256 (0xc076) ECDH x25519 (eq. 3072 bits RSA) FS **WEAK**

TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (0xc014) ECDH x25519 (eq. 3072 bits RSA) FS **WEAK**

TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (0xc013) ECDH x25519 (eq. 3072 bits RSA) FS **WEAK**

TLS_RSA_WITH_AES_256_GCM_SHA384 (0x9d) **WEAK**

TLS_RSA_WITH_AES_256_CCM_8 (0xc0a1) **WEAK**

TLS_RSA_WITH_AES_256_CCM (0xc09d) **WEAK**

TLS_RSA_WITH_ARIA_256_GCM_SHA384 (0xc051) **WEAK**

TLS_RSA_WITH_AES_128_GCM_SHA256 (0x9c) **WEAK**

TLS_RSA_WITH_AES_128_CCM_8 (0xc0a0) **WEAK**

TLS_RSA_WITH_AES_128_CCM (0xc09c) **WEAK**

TLS_RSA_WITH_ARIA_128_GCM_SHA256 (0xc050) **WEAK**

TLS_RSA_WITH_AES_256_CBC_SHA256 (0x3d) **WEAK**

TLS_RSA_WITH_CAMELLIA_256_CBC_SHA256 (0xc0) **WEAK**

TLS_RSA_WITH_AES_128_CBC_SHA256 (0x3c) **WEAK**

TLS_RSA_WITH_CAMELLIA_128_CBC_SHA256 (0xba) **WEAK**

TLS_RSA_WITH_AES_256_CBC_SHA (0x35) **WEAK**

TLS_RSA_WITH_CAMELLIA_256_CBC_SHA (0x84) **WEAK**

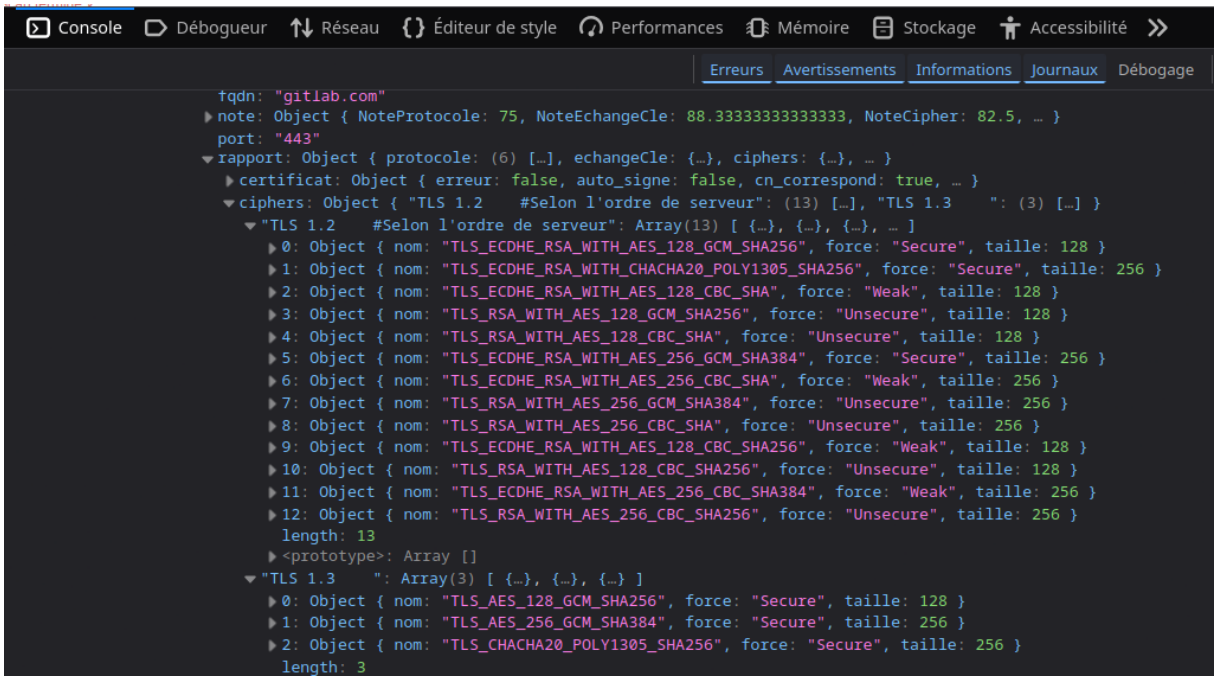
TLS_RSA_WITH_AES_128_CBC_SHA (0x2f) **WEAK**

TLS_RSA_WITH_CAMELLIA_128_CBC_SHA (0x41) **WEAK**

TLS 1.1 (suites in server-preferred order)

TLS 1.0 (suites in server-preferred order)

Test sur gitlab.com



TLS 1.3 (server has no preference)

TLS_AES_128_GCM_SHA256 (0x1301) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_AES_256_GCM_SHA384 (0x1302) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_CHACHA20_POLY1305_SHA256 (0x1303) ECDH x25519 (eq. 3072 bits RSA) FS

TLS 1.2 (suites in server-preferred order)

TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca8) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (0xc013) ECDH x25519 (eq. 3072 bits RSA) FS **WEAK**

TLS_RSA_WITH_AES_128_GCM_SHA256 (0x9c) **WEAK**

TLS_RSA_WITH_AES_128_CBC_SHA (0x2f) **WEAK**

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (0xc014) ECDH x25519 (eq. 3072 bits RSA) FS **WEAK**

TLS_RSA_WITH_AES_256_GCM_SHA384 (0x9d) **WEAK**

TLS_RSA_WITH_AES_256_CBC_SHA (0x35) **WEAK**

TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 (0xc027) ECDH x25519 (eq. 3072 bits RSA) FS **WEAK**

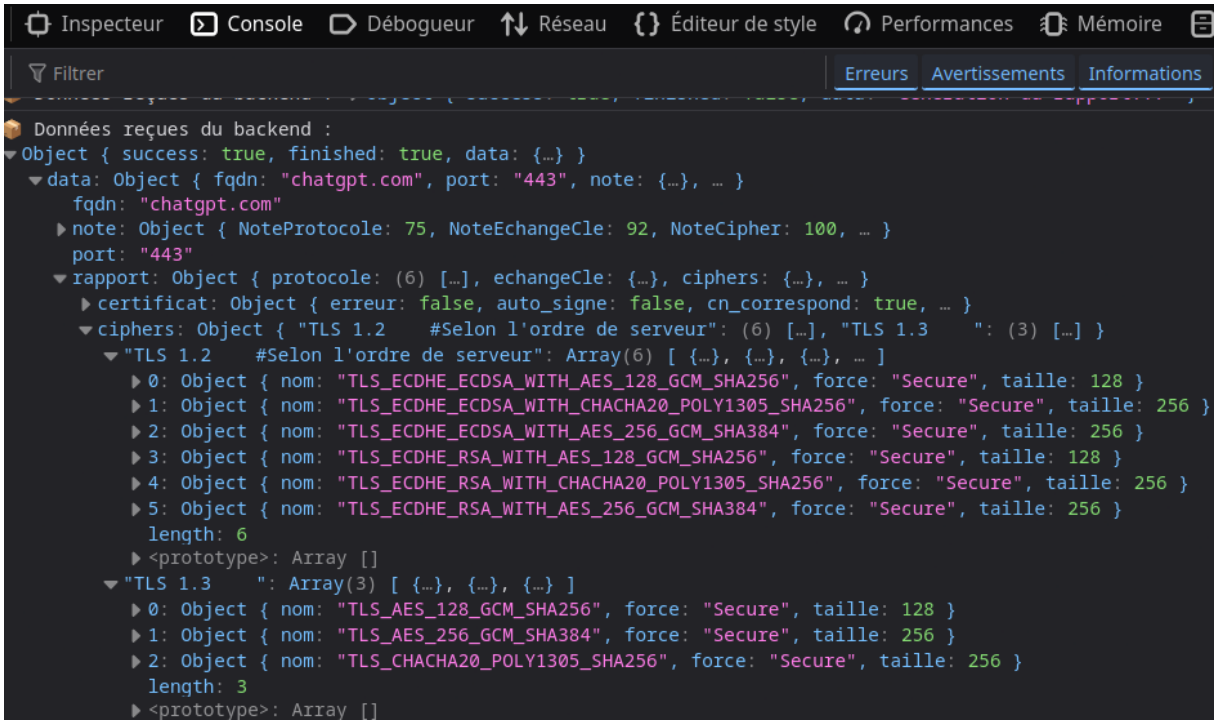
TLS_RSA_WITH_AES_128_CBC_SHA256 (0x3c) **WEAK**

TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384 (0xc028) ECDH x25519 (eq. 3072 bits RSA) FS **WEAK**

TLS_RSA_WITH_AES_256_CBC_SHA256 (0x3d) **WEAK**

(P) This server prefers ChaCha20 suites with clients that don't have AES-NI (e.g., Android devices)

Test sur chatgpt.com



Cipher Suites

TLS 1.3 (server has no preference)

TLS_AES_128_GCM_SHA256 (0x1301) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_AES_256_GCM_SHA384 (0x1302) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_CHACHA20_POLY1305_SHA256 (0x1303) ECDH x25519 (eq. 3072 bits RSA) FS

TLS 1.2 (suites in server-preferred order)

TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca9) ECDH x25519 (eq. 3072 bits RSA) FS

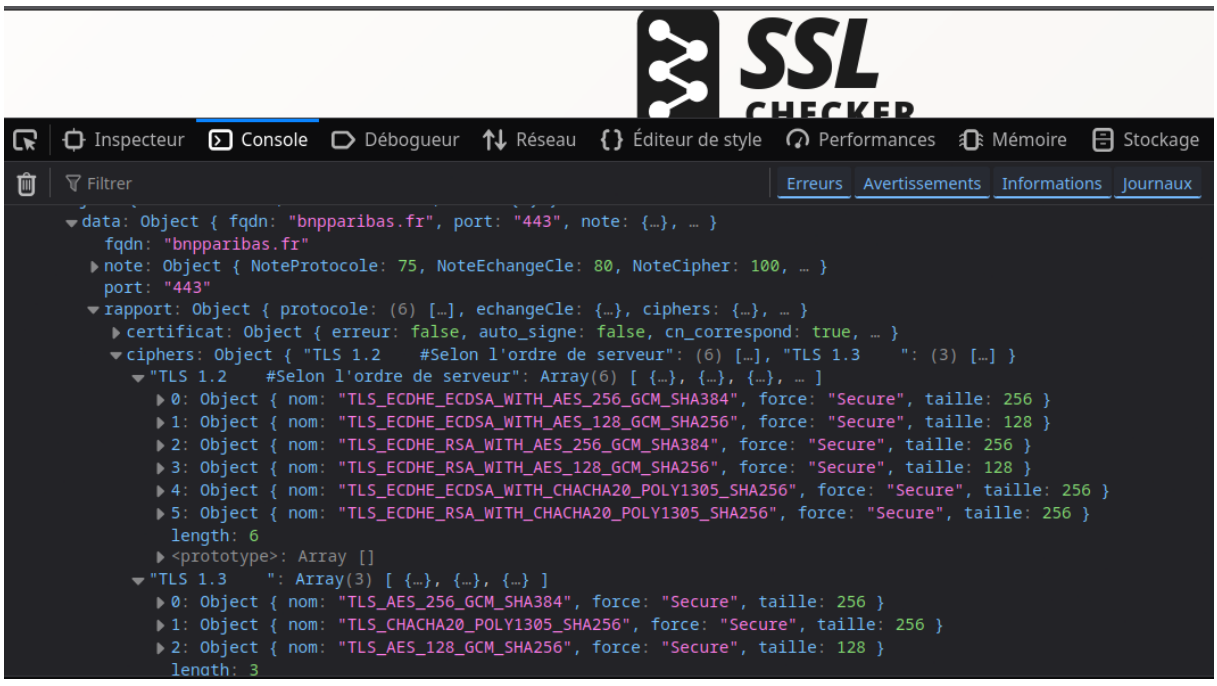
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (0xc02c) ECDH x25519 (eq. 3072 bits RSA) FS

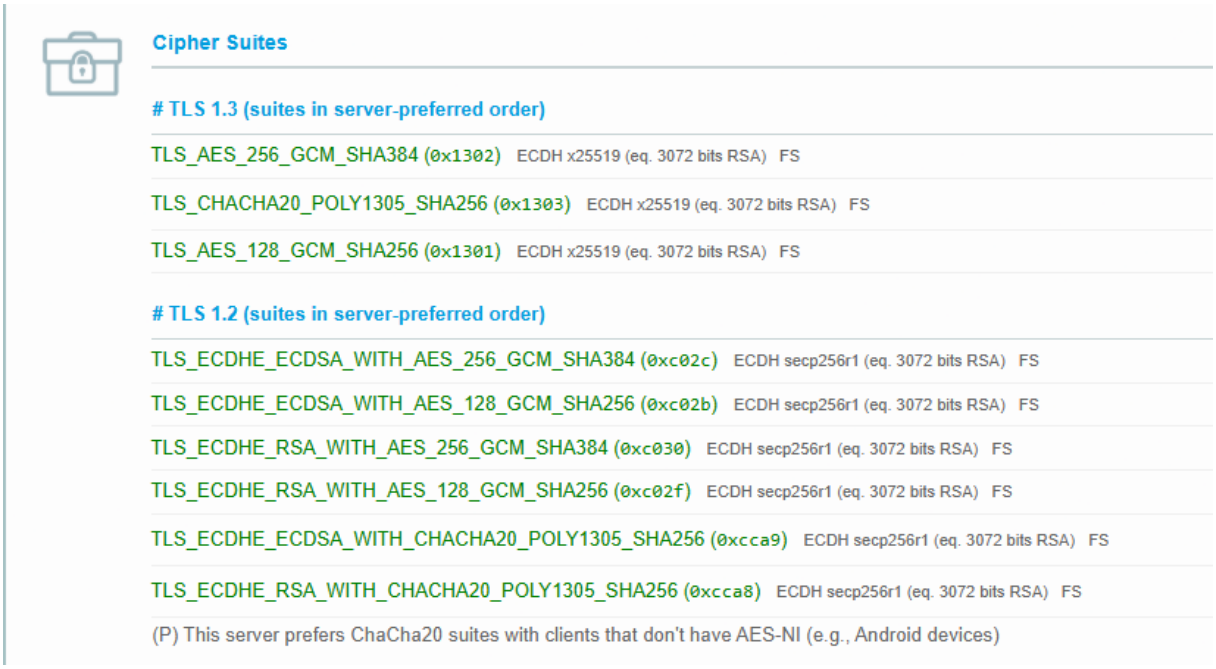
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca8) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030) ECDH x25519 (eq. 3072 bits RSA) FS

Test sur bnpparibas.fr





Cipher Suites

TLS 1.3 (suites in server-preferred order)

TLS_AES_256_GCM_SHA384 (0x1302) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_CHACHA20_POLY1305_SHA256 (0x1303) ECDH x25519 (eq. 3072 bits RSA) FS

TLS_AES_128_GCM_SHA256 (0x1301) ECDH x25519 (eq. 3072 bits RSA) FS

TLS 1.2 (suites in server-preferred order)

TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (0xc02c) ECDH secp256r1 (eq. 3072 bits RSA) FS

TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b) ECDH secp256r1 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030) ECDH secp256r1 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f) ECDH secp256r1 (eq. 3072 bits RSA) FS

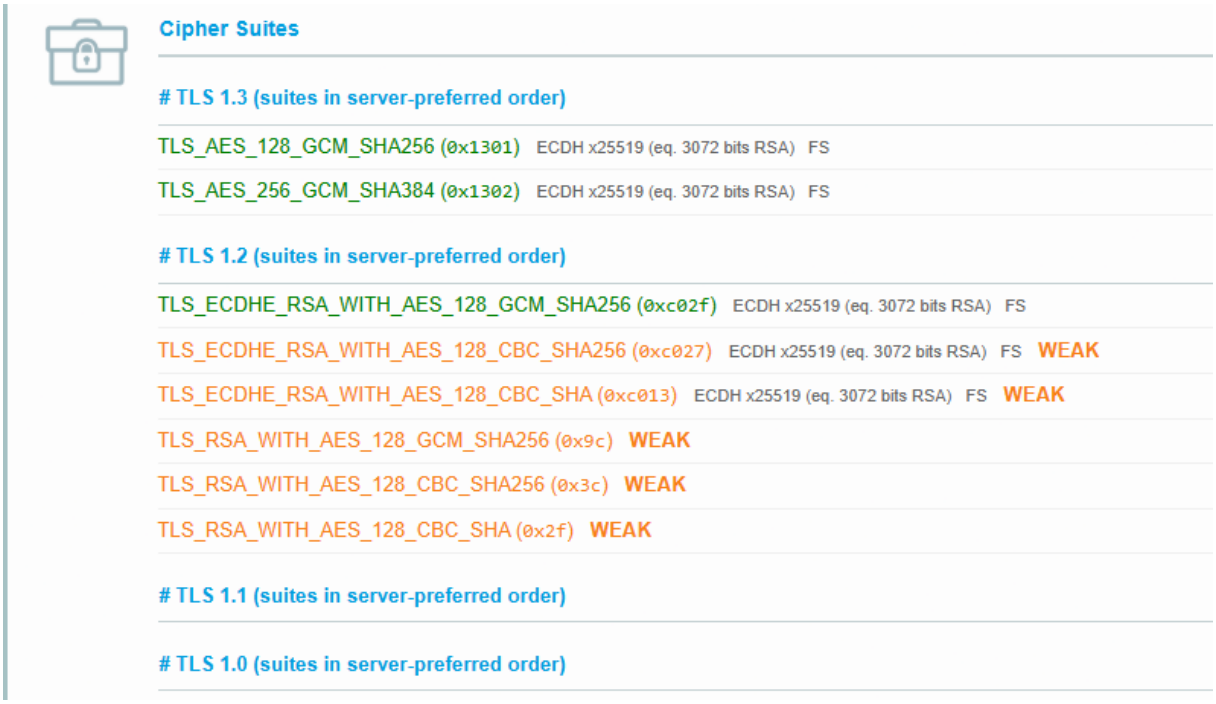
TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca9) ECDH secp256r1 (eq. 3072 bits RSA) FS

TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca8) ECDH secp256r1 (eq. 3072 bits RSA) FS

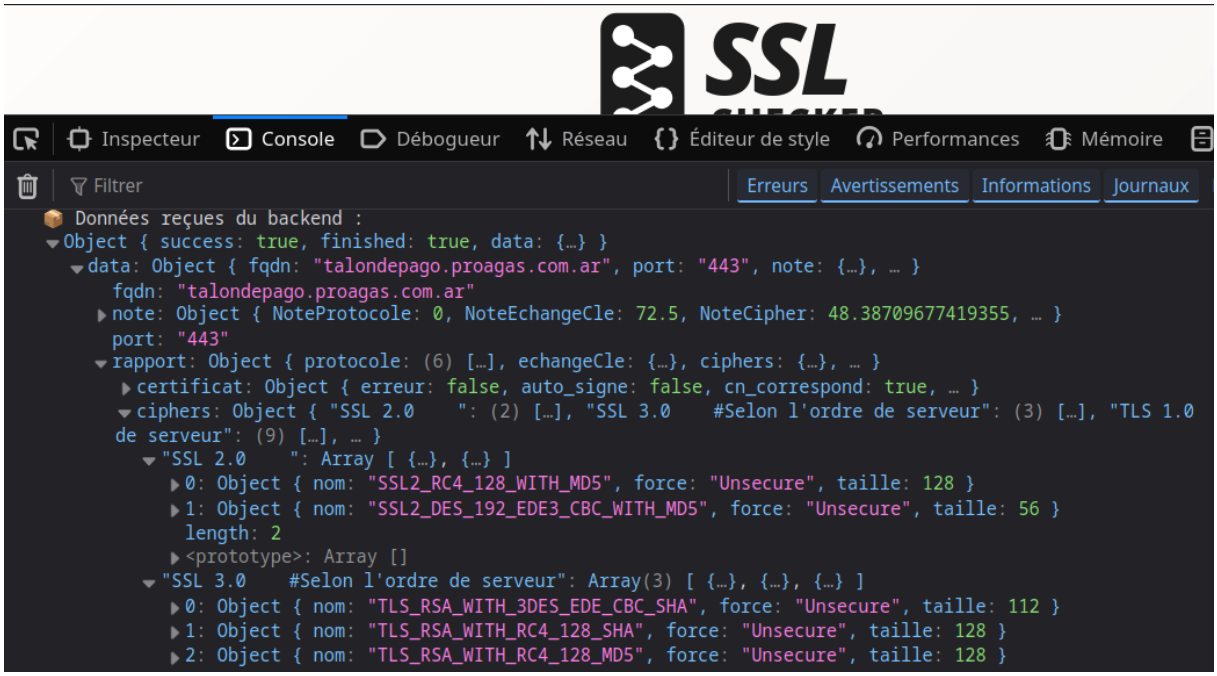
(P) This server prefers ChaCha20 suites with clients that don't have AES-NI (e.g., Android devices)

Test sur amazon.fr





1.1 Test sur des sites qui supportent des protocoles obsolètes SSLv2 et SSLv3



2 Module Certificat

2.1 Le cas parfait

C'est le scénario idéal où le serveur respecte toutes les normes de sécurité actuelles.

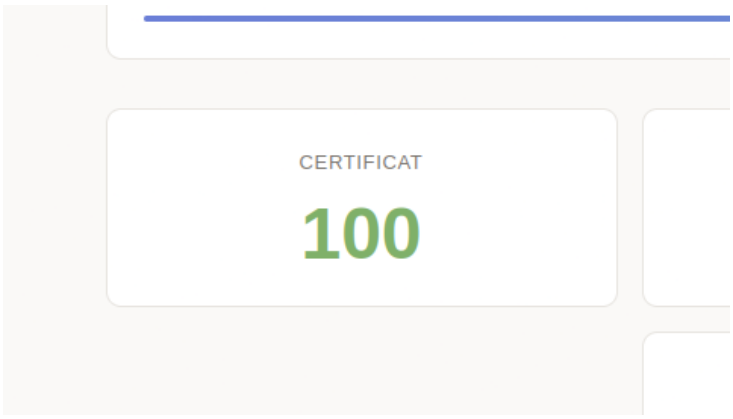
Domaine à tester : www.univ-rouen.fr (ou un site avec un certificat récent et valide). Ce que fait le code en arrière-plan :

- **Validité** : Le code vérifie que la date actuelle est bien comprise entre les dates de début et de fin du certificat. La fonction `_noter_validite` accorde 25/25 points si le certificat n'expire pas dans moins de 30 jours.
- **Autorité** : Le système confirme que le certificat est signé par une autorité reconnue et que le nom (CN ou SAN) correspond bien à l'adresse tapée. Aucune pénalité n'est appliquée.
- **Clé Publique** : Le code détecte une clé forte, comme une clé EC de 256 bits ou plus. Il attribue le maximum de points (jusqu'à 20/20).
- **Chaîne de confiance** : La chaîne est vérifiée comme étant "complète" jusqu'à la racine. Cela rapporte 25/25 points.
- **Signature & Révocation** : L'algorithme de signature est moderne (SHA-256) et le statut OCSP indique que le certificat n'est pas révoqué. Ces modules rapportent respectivement 15/15 et 5/5 points.

Résultat attendu

- Verdict Global : Un message "VALIDE" affiché en vert.
- Note Finale : Un score situé entre 90 et 100/100.
- Détails : Une liste de cases cochées ou d'icônes vertes pour chaque catégorie (Validité, Clé, Chaîne, etc.).





2.2 Certificat expiré

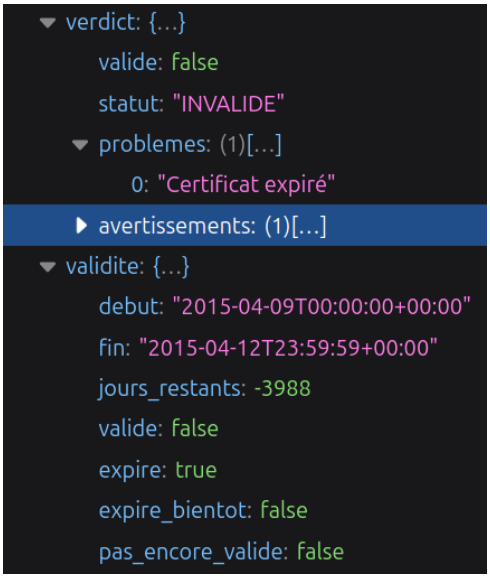
C'est le cas où la date de fin du certificat est passée.

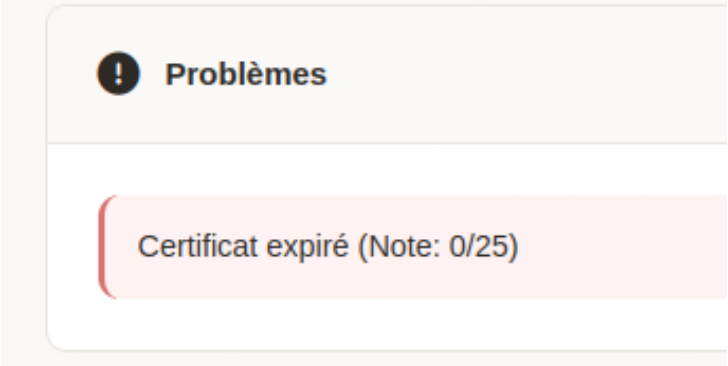
Domaine à tester : expired.badssl.com

Ce que fait le code : La fonction `_noter_validite` détecte que la date actuelle est supérieure à not_after et attribue une note de 0/25 pour la validité. Le verdict global passe immédiatement à INVALIDE.

Résultat espéré : Note éliminatoire

- Un message d'alerte rouge : **"Certificat expiré"**.
- L'affichage des dates montrant qu'il n'est plus valide.





2.3 Certificat auto-signé

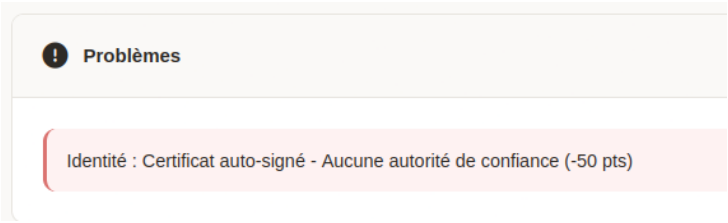
C'est un certificat qui n'a pas été validé par une autorité officielle.

- **Domaine à tester :** self-signed.badssl.com.
- **Ce que fait le code :** La fonction analyser_autorite voit que l'émetteur est le même que le sujet. La classe NoteCertificat applique alors une pénalité de -50 points.

— **Résultat espéré :** Le statut affiche INVALID donc une note nulle.
  — Un message indiquant "Certificat auto-signé".
  — Le score global est amputé de la moitié des points possibles.



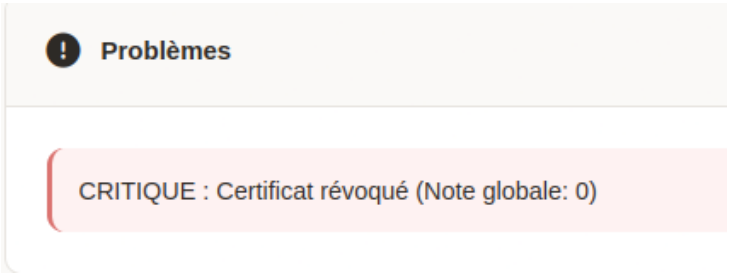




2.4 Certificat révoqué

Le certificat a été annulé par l'autorité de certification avant sa date d'expiration (par exemple, suite à un vol de clé).

— **Domaine à tester :** revoked.badssl.com.
— **Ce que fait le code :** La fonction analyser_revocation interroge les serveurs OCSP ou CRL. Si le statut renvoyé est "revoked", la note de révocation tombe à **0**.
— **Résultat espéré :** Statut **INVALIDÉ**.
  — Un message clair : **"Certificat révoqué"**



2.5 Signature faible (SHA-1 ou MD5)

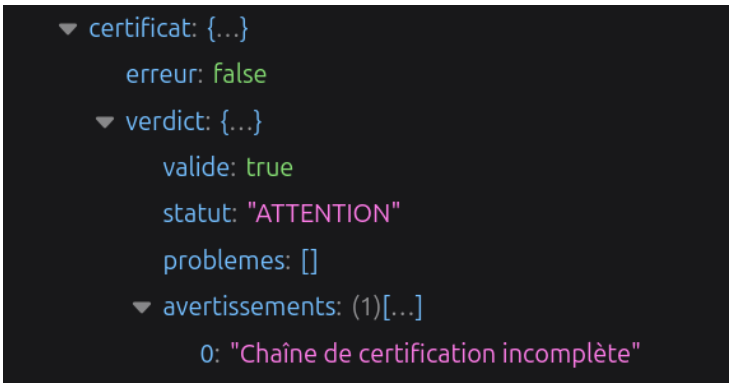
Le certificat utilise un algorithme de signature ancien et vulnérable aux attaques.

- **Domaine à tester :** Validation théorique par le code (Anciennement testé sur sha1-intermediate.badssl.com ou sha1-2048.badssl.com).
- **Note importante concernant ce test :** Il est aujourd'hui pratiquement impossible de trouver des sites publics en ligne (même des sites de test) utilisant encore SHA-1 ou MD5, car ces algorithmes ont été universellement bannis et bloqués par les navigateurs modernes. Par conséquent, il n'est plus possible de fournir une cible "en direct" pour ce cas précis.
- **Ce que fait le code :** Malgré l'absence de sites vulnérables en ligne pour tester, **notre application est parfaitement configurée pour bloquer ces failles**. La fonction analyser_signature est programmée pour vérifier tous les maillons de la chaîne de certification. Si elle identifie les termes sha1 ou md5 (y compris dans un certificat intermédiaire), elle passe le statut de sécurité à False et la note de signature tombe immédiatement à **0/15**.
- **Résultat théorique espéré :** Statut **INVALIDE**.
  - Message : **"Algorithme de signature faible (sha1)"**.

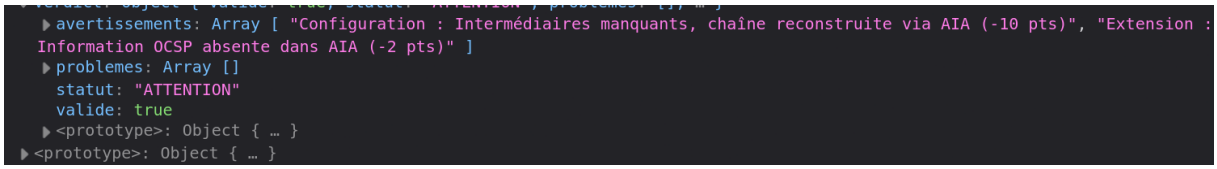
2.6 Chaîne de certification incomplète

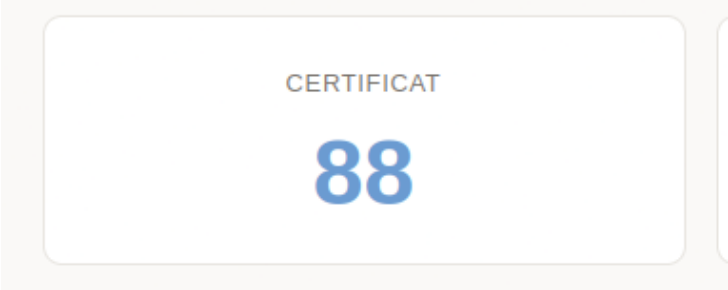
Le serveur n'envoie pas tous les certificats intermédiaires nécessaires pour remonter jusqu'à la racine de confiance.

- **Domaine à tester :** incomplete-chain.badssl.com.
- **Ce que fait le code :** Le code tente de reconstruire la chaîne. S'il n'y parvient pas totalement, complete est mis à False. La note de la chaîne est réduite à **15/25**.
- **Résultat espéré :** Statut **ATTENTION**.
  - Message : **"Chaîne de certification incomplète"**.



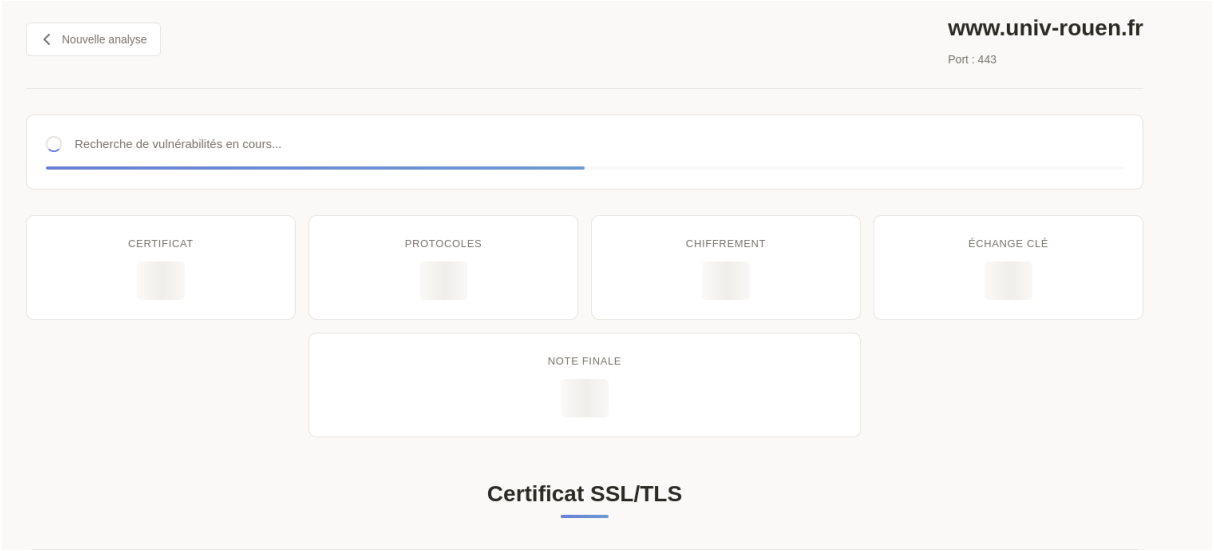
Après Amélioration nous avons intégré un mécanisme qui force à retrouver la chaîne de certificat même quand elle n'est pas renvoyée par le serveur, bien évidemment cela retirera des points à la note de certificat pour rester à 15/25. On aura donc comme message : **"Le serveur n'envoie pas les intermédiaires (chaîne reconstruite via AIA)"**.

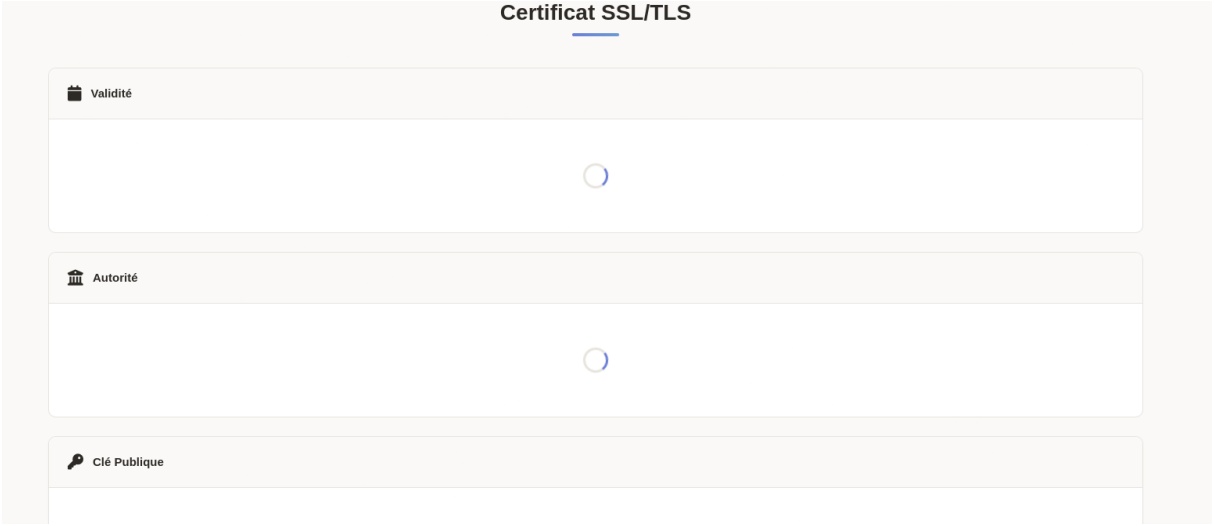




3 Le Front-end

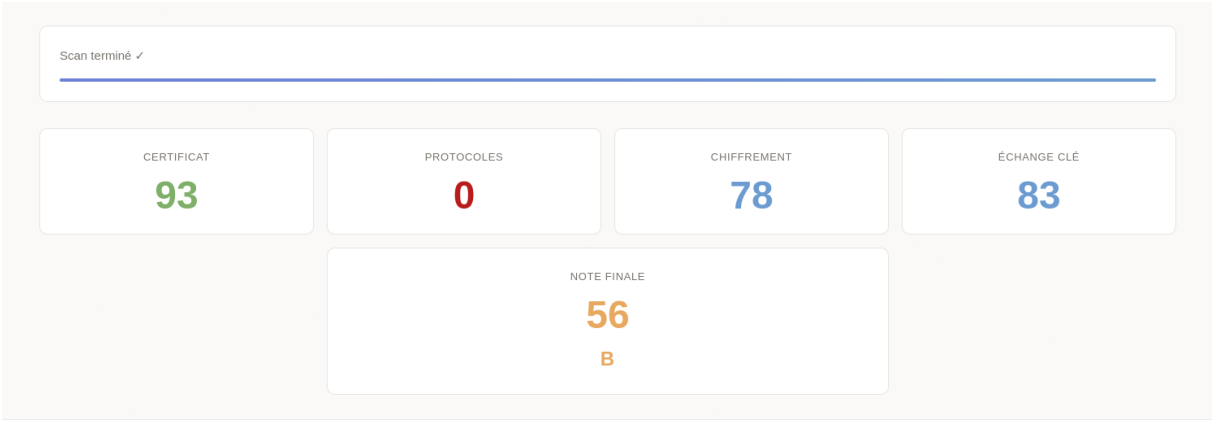
1. Le "Skeleton Screen" (États de chargement)
   - **Description :** Vérifier que l'interface ne reste pas vide ou "cassée" pendant la première seconde où le front attend les données du back-end via SSE.
   - **Action à tester :** Lancer une analyse valide (ex : google.com) et observer l'écran avant que la jauge n'atteigne 100%.
   - **Ce que fait le code (Front) :** La fonction resetDashboard() masque les valeurs textuelles et force l'affichage de l'élément .summary-skeleton (qui contient l'animation CSS @keyframes shimmer) et injecte la div .spinner-ring dans toutes les cartes de données.
   - **Résultat espéré (UI) :** Les 4 cartes de score affichent un bloc gris avec une animation de balayage lumineuse fluide. Toutes les boîtes du bas (Certificat, Protocoles...) affichent un cercle de chargement animé et parfaitement centré.



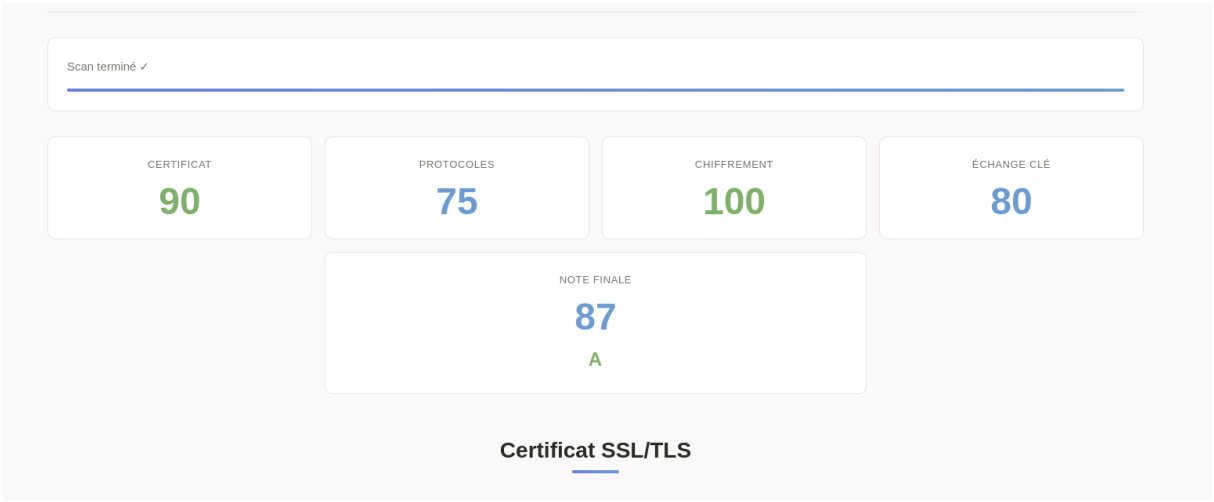


2. **L'Écrasement Total du Dashboard (Reset State)**
   - **Description :** Vérifier que lancer un deuxième scan nettoie parfaitement toutes les données du premier, pour ne pas afficher des résultats mélangés.
   - **Action à tester :** Lancer un premier scan qui a des vulnérabilités (beaucoup de rouge). Attendre la fin. Cliquer sur "Nouvelle analyse" (ou relancer direct), et analyser un domaine parfait (beaucoup de vert).
   - **Ce que fait le code (Front) :** Avant chaque nouveau scan, UI.resetDashboard() boucle sur DOM.content et remplace le contenu de chaque zone par la constante loader. Il supprime aussi les classes CSS de couleurs (excellent, poor, etc.) sur les cartes de score.
   - **Résultat espéré (UI) :** Aucune ancienne donnée (texte, badge rouge, commentaire) ne "clignote" ou ne reste affichée. L'interface redevient vierge et neutre instantanément avant l'arrivée des nouvelles données.

Avant (le test avec la couleur rouge)

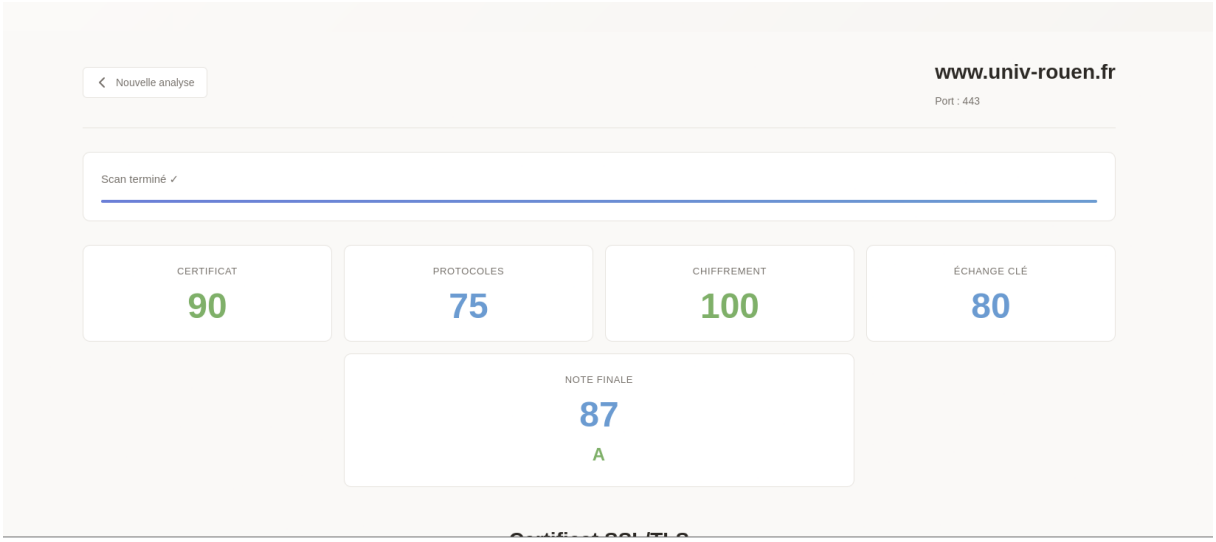


Après (une nouvelle analyse)

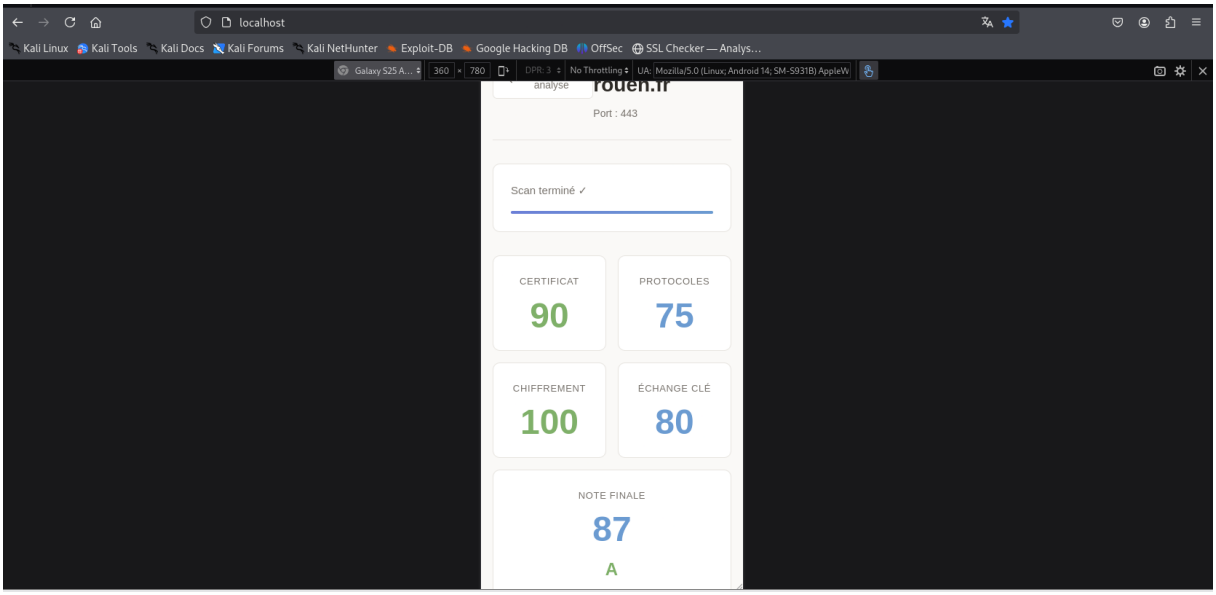


3. **Le Responsive "Breakpoint" (Design Mobile)**
   - **Description :** Tester la transition du design "Bureau" vers le design "Smartphone" pour éviter l'apparition d'un scroll horizontal désagréable. Action à tester : Ouvrir le dashboard complet à 100% avec des données. Réduire lentement la largeur de la fenêtre du navigateur jusqu'à atteindre une largeur de smartphone (ex : 375px).
   - **Ce que fait le code (Front) :** Le CSS contient une Media Query @media (max-width : 768px) qui modifie la grille. La .summary-grid passe de 4 colonnes à 2 colonnes (grid-template-columns : 1fr 1fr ;), et .section-grid passe sur 1 seule colonne.
   - **Résultat espéré (UI) :** Les 4 cartes de score s'affichent par rangées de 2. La carte "Note Finale" (.summary-card-global) prend toute la largeur de l'écran (100%). Les cartes de données s'empilent verticalement, sans aucun débordement horizontal.

Test plein écran :



Test écran téléphone :

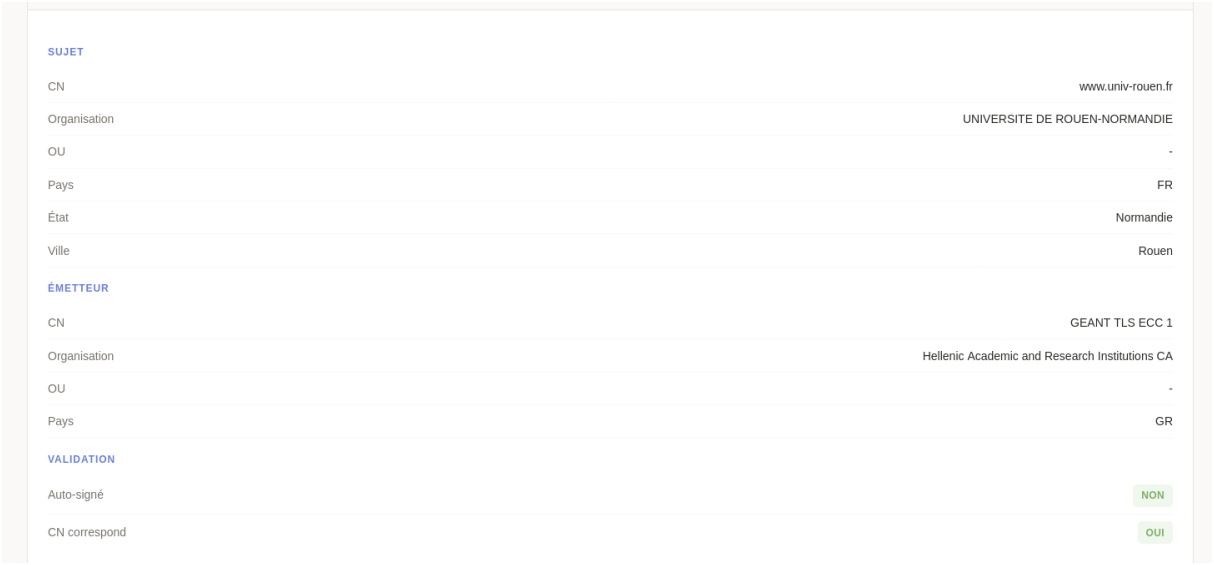


4. **Le Rendu des Champs Vides (Empty States Fallbacks)**
   - **Description :** Vérifier comment le front-end gère l'absence d'une information textuelle spécifique (comme une Unité Organisationnelle "OU" manquante) au lieu d'afficher des erreurs comme undefined ou de laisser un vide perturbant pour l'œil.
   - **Action à tester :** Analyser un domaine dont le certificat ne possède pas tous les champs (ex : pas de OU), ou forcer l'objet cert.sujet.OU à undefined ou "" (chaîne vide).
   - **Ce que fait le code (Front) :** La fonction renderCert utilise le helper infoRow combiné à l'opérateur logique ou court-circuit : infoRow('OU', sub.OU || '-').

Si la valeur n'existe pas ou est vide, elle assigne automatiquement le tiret en argument.

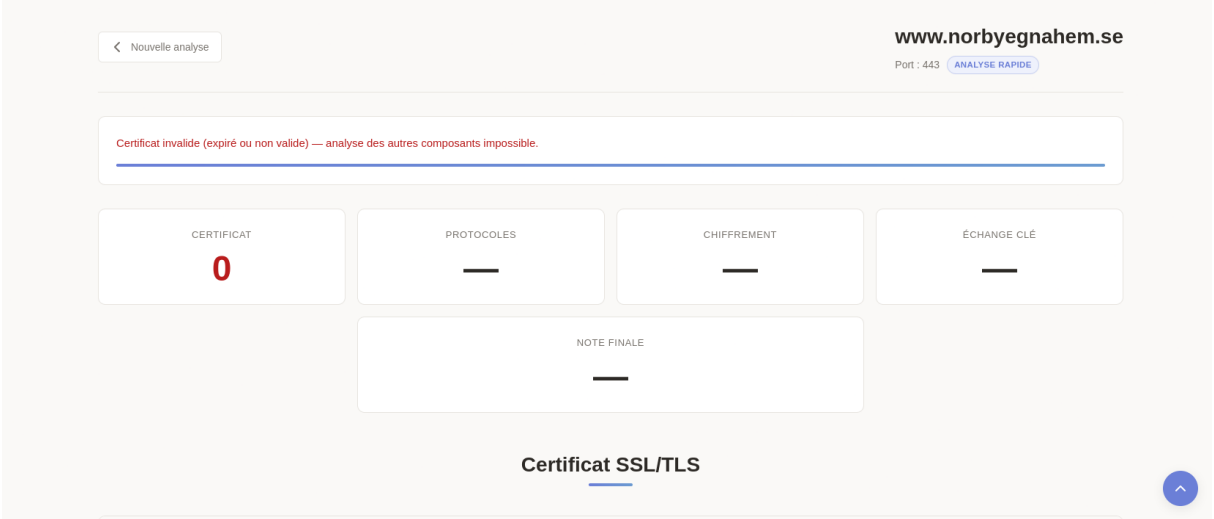
— **Résultat espéré (UI) :** Au lieu d'avoir un trou blanc sur la ligne, l'interface affiche proprement un tiret (-) aligné à droite. Cela confirme visuellement à l'utilisateur que l'outil a bien lu le certificat, mais que la donnée est officiellement absente côté serveur.

Preuve :



5. **L'Arrêt Prématuré (Gestion des Certificats Expirés)**
   — **Description :** Vérifier que le front-end arrête intelligemment le rendu et ne lance pas de chargements infinis si le certificat s'avère invalide ou expiré (il est inutile de tester le chiffrement sur un certificat mort).
   — **Action à tester :** Scanner un domaine dont le certificat est expiré (ex : expired.badssl.com).
   — **Ce que fait le code (Front) :** La fonction renderAll() vérifie la variable isCertValid en tout premier. Si elle est fausse, le code masque intégralement toutes les boîtes .data-sections .section (à l'exception de celle du certificat), affiche un message d'erreur rouge dans la barre de statut, force la note du certificat à 0 et applique la fonction setScoreDash() pour mettre les autres notes à —
   — **Résultat espéré (UI) :** L'analyse s'arrête net. Seule la carte "Certificat SSL/TLS" est visible. Aucun "spinner" de chargement ne reste coincé sur l'écran. Les scores affichent un gros "0" rouge pour le certificat, et un tiret gris pour les autres catégories. *(Espace pour tes captures d'écran)*

Preuve :



4 Module Protocole

On teste toutes les versions : **SSL 2**, **SSL 3**, **TLS 1.0**, **TLS 1.1**, **TLS 1.2**, **TLS 1.3**.

On distingue deux catégories de tests : **SSL** et **TLS**. La raison est la suivante : des serveurs de référence pour tester les protocoles **TLS** sont disponibles publiquement, alors que des serveurs pour **SSL** ne le sont pas facilement. C'est pourquoi nous avons choisi de créer un conteneur Docker configuré pour supporter **SSL 2** ou **SSL 3**.

4.1 Test sur les protocoles TLS

```
root@043bb0257b19:/app# python -c "
> from classes.tests.testProtocole import TestProtocole
> t = TestProtocole('google.com', '443')
> t.lancer()
> print(t.getResultat())
> "
[{'nom': 'SSL 2.0', 'supporte': False}, {'nom': 'SSL 3.0', 'supporte': False}, {'nom': 'TLS 1.0', 'supporte': True}, {'nom
': 'TLS 1.1', 'supporte': True}, {'nom': 'TLS 1.2', 'supporte': True}, {'nom': 'TLS 1.3', 'supporte': True}]
root@043bb0257b19:/app# python -c "
> from classes.tests.testProtocole import TestProtocole
> t = TestProtocole('tls-v1-2.badssl.com', '1012')
> t.lancer()
> print(t.getResultat())
> "
[{'nom': 'SSL 2.0', 'supporte': False}, {'nom': 'SSL 3.0', 'supporte': False}, {'nom': 'TLS 1.0', 'supporte': False}, {'no
m': 'TLS 1.1', 'supporte': False}, {'nom': 'TLS 1.2', 'supporte': True}, {'nom': 'TLS 1.3', 'supporte': False}]
root@043bb0257b19:/app# python -c "
> from classes.tests.testProtocole import TestProtocole
> t = TestProtocole('tls-v1-1.badssl.com', '1011')
> t.lancer()
> print(t.getResultat())
> "
[{'nom': 'SSL 2.0', 'supporte': False}, {'nom': 'SSL 3.0', 'supporte': False}, {'nom': 'TLS 1.0', 'supporte': False}, {'no
m': 'TLS 1.1', 'supporte': True}, {'nom': 'TLS 1.2', 'supporte': False}, {'nom': 'TLS 1.3', 'supporte': False}]
root@043bb0257b19:/app# python -c "
> from classes.tests.testProtocole import TestProtocole
> t = TestProtocole('tls-v1-0.badssl.com', '1010')
> t.lancer()
> print(t.getResultat())
> "
[{'nom': 'SSL 2.0', 'supporte': False}, {'nom': 'SSL 3.0', 'supporte': False}, {'nom': 'TLS 1.0', 'supporte': True}, {'nom
': 'TLS 1.1', 'supporte': False}, {'nom': 'TLS 1.2', 'supporte': False}, {'nom': 'TLS 1.3', 'supporte': False}]
root@043bb0257b19:/app#
```

4.2 Test sur les protocoles SSL

Pour faire le test, on a deux fichiers qui seront utilisés : `docker-compose.test.yml` et `Dockerfile.legacy`.
Pour `docker-compose.test.yml`, on a :

docker-compose.test.yml

```yaml
# docker-compose.test.yml
version: '3.8'

services:
  ssl_legacy:
    container_name: ssl_legacy_server
    build:
      context: .
      dockerfile: Dockerfile.legacy
    ports:
      - "4443:443"
```

Pour `Dockerfile.legacy`, il est préparé selon la version à tester.

— SSL 2

```
Dockerfile.legacy > ...
1   an:jessie
2
3   "deb http://archive.debian.org/debian jessie main" > /etc/apt/sources.list && \
4   "deb http://archive.debian.org/debian-security jessie/updates main" >> /etc/apt/sources.list
5
6   et update -o Acquire::Check-Valid-Until=false && apt-get install -y --force-yes \
7   uild-essential \
8   erl \
9    -rf /var/lib/apt/lists/*
10
11  ssl-1.0.2u.tar.gz .
12
13  xzf openssl-1.0.2u.tar.gz && \
14  enssl-1.0.2u && \
15  fig --prefix=/opt/openssl enable-ssl2 enable-ssl3 && \
16  && make install && \
17   && rm -rf openssl-1.0.2u openssl-1.0.2u.tar.gz
18
19  openssl/bin/openssl req -x509 -newkey rsa:1024 -keyout /key.pem -out /cert.pem -days 365 -nodes \
20   "/CN=ssl-legacy-test"
21
22  openssl/bin/openssl s_server -key /key.pem -cert /cert.pem -accept 443 -ssl2 -www
```

— SSL 3

```
Dockerfile.legacy > ...
1    debian:jessie
2
3   inter vers les dépôts archivés
4   echo "deb http://archive.debian.org/debian jessie main" > /etc/apt/sources.list && \
5   echo "deb http://archive.debian.org/debian-security jessie/updates main" >> /etc/apt/sources.list
6
7   apt-get update -o Acquire::Check-Valid-Until=false && apt-get install -y --force-yes \
8       openssl \
9   && rm -rf /var/lib/apt/lists/*
10
11  nérer un certificat auto-signé
12  openssl req -x509 -newkey rsa:2048 -keyout /key.pem -out /cert.pem -days 365 -nodes \
13  -subj "/CN=ssl-legacy-test"
14
15  ncer un serveur SSL avec SSL 2.0/3.0 activés
16  openssl s_server -key /key.pem -cert /cert.pem -accept 443 -ssl3 -www
```

Alors, pour les résultats des tests, pour SSL 2 et SSL 3, on obtient :

— **SSL 2**

```
root@11c328546519:/app# python -c "
> from classes.tests.testProtocole import TestProtocole
> t = TestProtocole('ssl_legacy_server', '443')
> t.lancer()
> print(t.getResultat())
> "
[{'nom': 'SSL 2.0', 'supporte': True}, {'nom': 'SSL 3.0', 'supporte': False}, {'nom': 'TLS 1.0', 'supporte': False}, {'nom
': 'TLS 1.1', 'supporte': False}, {'nom': 'TLS 1.2', 'supporte': False}, {'nom': 'TLS 1.3', 'supporte': False}]
root@11c328546519:/app#
```

— **SSL 3**

```
root@043bb0257b19:/app# python -c "
> from classes.tests.testProtocole import TestProtocole
> t = TestProtocole('ssl_legacy_server', '443')
> t.lancer()
> print(t.getResultat())
> "
[{'nom': 'SSL 2.0', 'supporte': False}, {'nom': 'SSL 3.0', 'supporte': True}, {'nom': 'TLS 1.0', 'supporte': Fa
lse}, {'nom': 'TLS 1.1', 'supporte': False}, {'nom': 'TLS 1.2', 'supporte': False}, {'nom': 'TLS 1.3', 'support
e': False}]
root@043bb0257b19:/app#
```

5 Module Compatibilité OS

Pour faire nos tests, on a choisi **pytest**, on a donc utilisé `test_testCompatibiliteOs_unit.py` pour faire nos tests, qui ont pour objectif de vérifier :

— que la fonction `_est_compatible` de testCompatibiliteOs fonctionne correctement

— que la fonction `lancer` de testCompatibiliteOs fonctionne correctement

Voici alors le code de `test_testCompatibiliteOs_unit.py` :

```python
def test_protocole_et_cipher_incompatibles(self, instance):
    """Ni protocole ni cipher en commun → False"""
    client = {
        "lowestProtocol": 768,   # SSL 3.0
        "highestProtocol": 769,  # TLS 1.0
        "suiteIds": [4, 5]
    }
    protocoles_serveur = {"TLS 1.2", "TLS 1.3"}
    ciphers_serveur = {4865, 4866}

    assert instance._est_compatible(client, protocoles_serveur, ciphers_serveur) is False

def test_protocoles_serveur_vide(self, instance):
    """Serveur sans protocoles supportés → False"""
    client = {
        "lowestProtocol": 769,
        "highestProtocol": 772,
        "suiteIds": [4865]
    }
    protocoles_serveur = set()
    ciphers_serveur = {4865}

    assert instance._est_compatible(client, protocoles_serveur, ciphers_serveur) is False

def test_ciphers_serveur_vide(self, instance):
    """Serveur sans ciphers supportées → False"""
    client = {
        "lowestProtocol": 771,
        "highestProtocol": 772,
        "suiteIds": [4865]
    }
    protocoles_serveur = {"TLS 1.2", "TLS 1.3"}
    ciphers_serveur = set()

    assert instance._est_compatible(client, protocoles_serveur, ciphers_serveur) is False

def test_client_sans_suite_ids(self, instance):
    """Client sans suiteIds → False"""
    client = {
        "lowestProtocol": 771,
        "highestProtocol": 772,
    }
    protocoles_serveur = {"TLS 1.2", "TLS 1.3"}
    ciphers_serveur = {4865, 4866}

    assert instance._est_compatible(client, protocoles_serveur, ciphers_serveur) is False
```

```python
class TestLancer:

    def test_lancer_produit_liste(self, instance):
        """lancer() produit une liste de dicts avec nom et compatible"""
        clients_mock = [
            {
                "name": "Android",
                "version": "9.0",
                "platform": "",
                "lowestProtocol": 769,
                "highestProtocol": 772,
                "suiteIds": [4865, 4866]
            }
        ]
        instance._clients = clients_mock

        protocoles = [
            {"nom": "TLS 1.2", "supporte": True},
            {"nom": "TLS 1.3", "supporte": True},
        ]
        ciphers = {
            "TLS 1.3": [(4865, "TLS_AES_128_GCM_SHA256")]
        }

        instance.lancer(protocoles, ciphers)
        resultat = instance.getResultat()

        assert isinstance(resultat, list)
        assert len(resultat) == 1
        assert resultat[0]["nom"] == "Android 9.0"
        assert resultat[0]["compatible"] is True
```

```python
def test_lancer_client_incompatible(self, instance):
    """lancer() détecte correctement un client incompatible"""
    clients_mock = [
        {
            "name": "IE",
            "version": "6",
            "platform": "XP",
            "lowestProtocol": 768,
            "highestProtocol": 768,
            "suiteIds": [4, 5]
        }
    ]
    instance._clients = clients_mock

    protocoles = [
        {"nom": "TLS 1.2", "supporte": True},
        {"nom": "TLS 1.3", "supporte": True},
    ]
    ciphers = {
        "TLS 1.3": [(4865, "TLS_AES_128_GCM_SHA256")]
    }

    instance.lancer(protocoles, ciphers)
    resultat = instance.getResultat()

    assert resultat[0]["nom"] == "IE 6 / XP"
    assert resultat[0]["compatible"] is False

def test_lancer_protocoles_vides(self, instance):
    """lancer() avec protocoles vides → tous incompatibles"""
    clients_mock = [
        {
            "name": "Chrome",
            "version": "80",
            "platform": "Win 10",
            "lowestProtocol": 771,
            "highestProtocol": 772,
            "suiteIds": [4865]
        }
    ]
    instance._clients = clients_mock

    instance.lancer([], {})
    resultat = instance.getResultat()

    assert resultat[0]["compatible"] is False
```

On obtient donc le résultat suivant :

```
PS C:\Users\Malik\Documents\sslchecker-dev\sslchecker-git-8\sslchecker-dev> python -m pytest -v app/classes/tests/test_tes
tCompatibiliteOs_unit.py
============================================= test session starts =============================================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Malik\AppData\Local\Python\pythoncore-3.14-64\pyth
on.exe
cachedir: .pytest_cache
rootdir: C:\Users\Malik\Documents\sslchecker-dev\sslchecker-git-8\sslchecker-dev
collected 10 items

app/classes/tests/test_testCompatibiliteOs_unit.py::TestEstCompatible::test_client_compatible PASSED               [ 10%]
app/classes/tests/test_testCompatibiliteOs_unit.py::TestEstCompatible::test_protocole_incompatible PASSED          [ 20%]
app/classes/tests/test_testCompatibiliteOs_unit.py::TestEstCompatible::test_cipher_incompatible PASSED             [ 30%]
app/classes/tests/test_testCompatibiliteOs_unit.py::TestEstCompatible::test_protocole_et_cipher_incompatibles PASSED [ 40%
]
app/classes/tests/test_testCompatibiliteOs_unit.py::TestEstCompatible::test_protocoles_serveur_vide PASSED         [ 50%]
app/classes/tests/test_testCompatibiliteOs_unit.py::TestEstCompatible::test_ciphers_serveur_vide PASSED           [ 60%]
app/classes/tests/test_testCompatibiliteOs_unit.py::TestEstCompatible::test_client_sans_suite_ids PASSED          [ 70%]
app/classes/tests/test_testCompatibiliteOs_unit.py::TestLancer::test_lancer_produit_liste PASSED                  [ 80%]
app/classes/tests/test_testCompatibiliteOs_unit.py::TestLancer::test_lancer_client_incompatible PASSED            [ 90%]
app/classes/tests/test_testCompatibiliteOs_unit.py::TestLancer::test_lancer_protocoles_vides PASSED               [100%]

============================================== warnings summary ===============================================
app\classes\tests\testCompatibiliteOs.py:18
  C:\Users\Malik\Documents\sslchecker-dev\sslchecker-git-8\sslchecker-dev\app\classes\tests\testCompatibiliteOs.py:18: Pyt
estCollectionWarning: cannot collect test class 'TestCompatibiliteOs' because it has a __init__ constructor (from: app/cla
sses/tests/test_testCompatibiliteOs_unit.py)
    class TestCompatibiliteOs(TestBase):

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================================== 10 passed, 1 warning in 0.09s ==========================================
PS C:\Users\Malik\Documents\sslchecker-dev\sslchecker-git-8\sslchecker-dev>
```

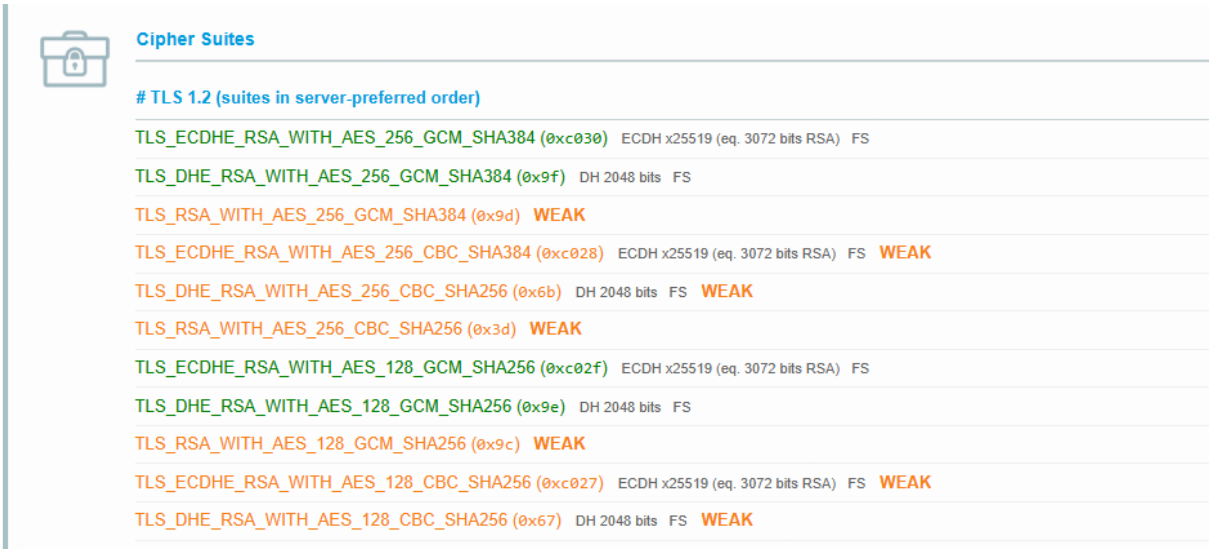
Ce test montre que les principaux cas de compatibilité sont correctement gérés. D'autres scénarios simples restent à tester ultérieurement : clients sans cipher, clients avec protocoles très anciens ou configurations atypiques.

6 Module Échange de clé

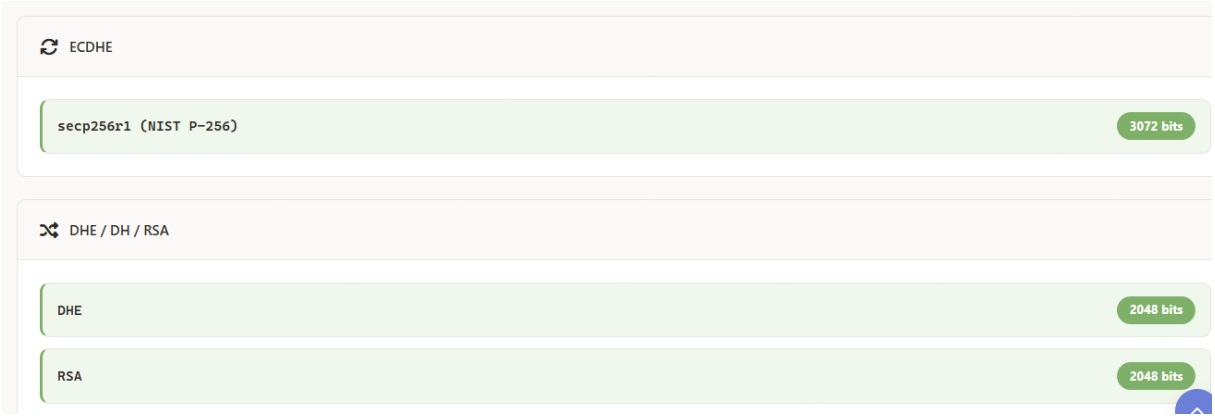
Dans cette section, nous allons tester le module d'échange de clés sur des sites présentant des clés robustes et d'autres dont les clés sont obsolètes. Afin de vérifier la cohérence des résultats, nous les avons comparés avec ceux retournés par SSL Labs, outil réputé pour sa fiabilité.

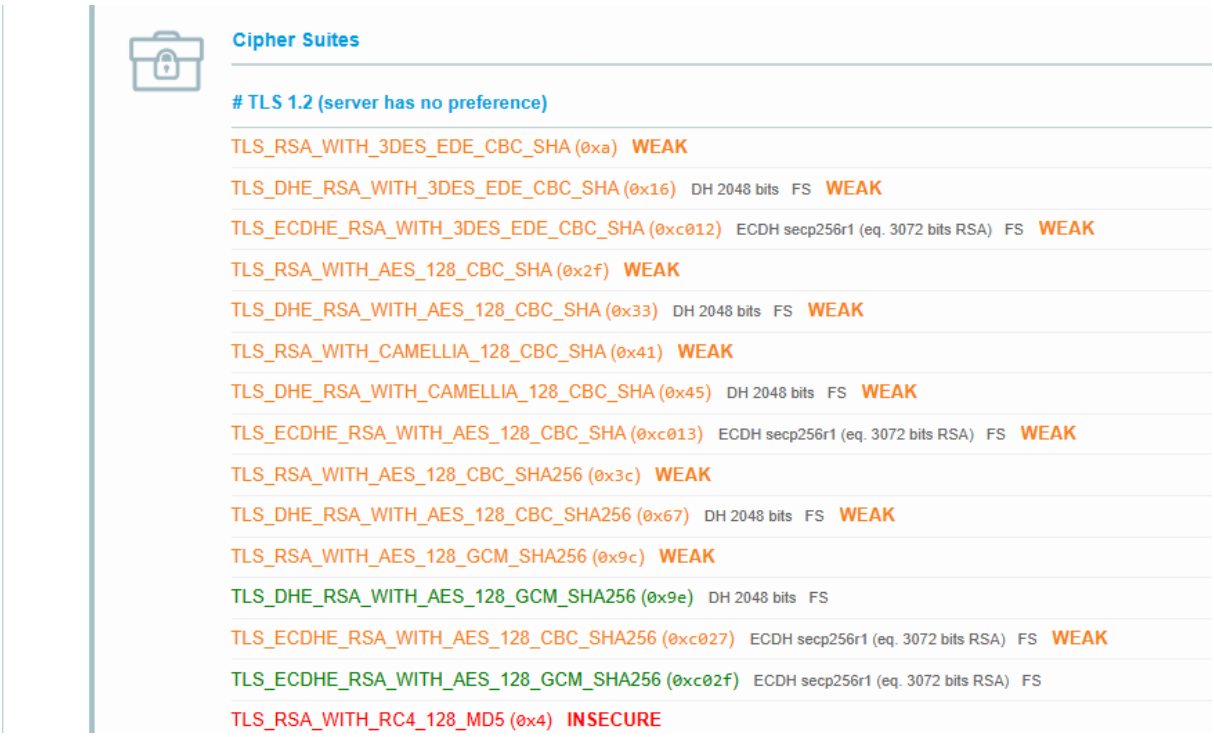
6.1 Test sur vpn2.gulfstream.ru





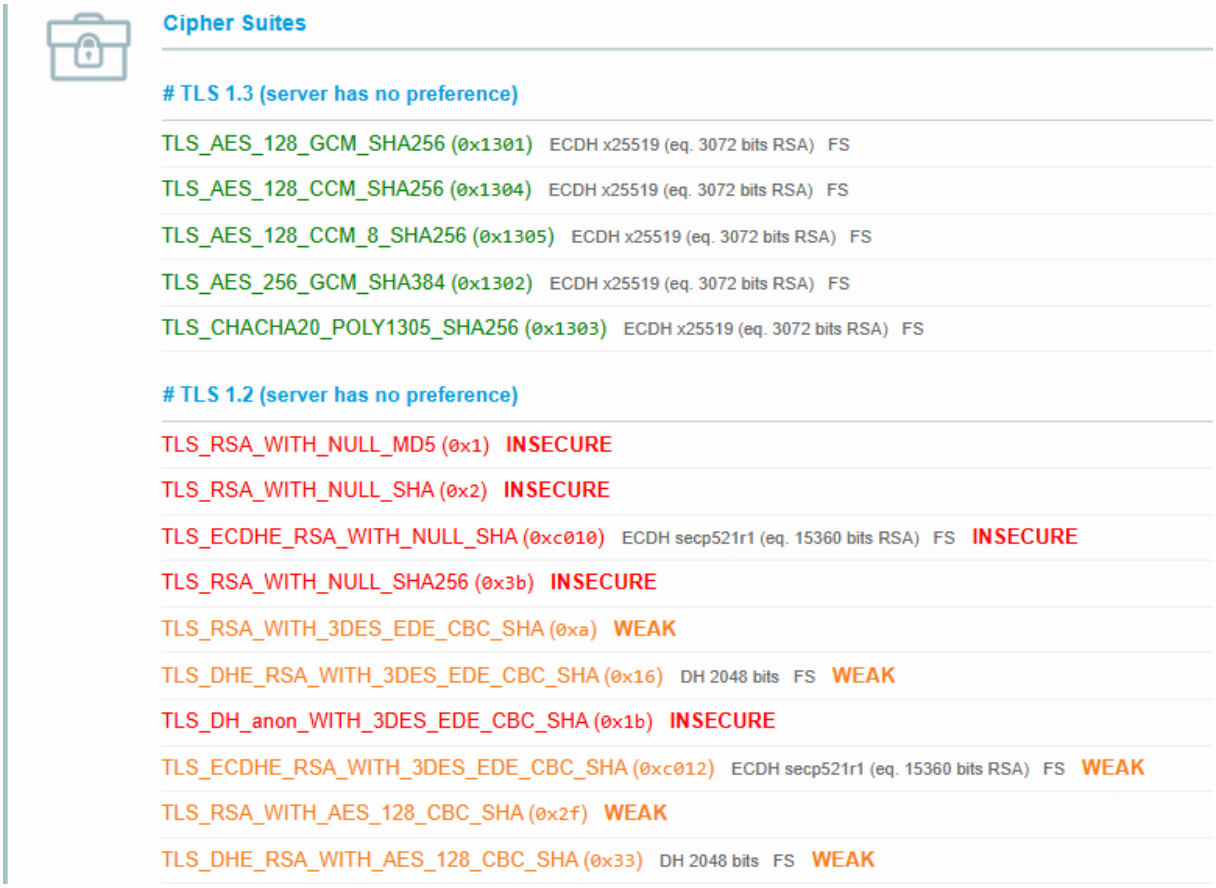
6.2 Test sur mail.anafra.cz





6.3 Test sur eminwon.haman.go.kr





7 Module Vulnérabilités

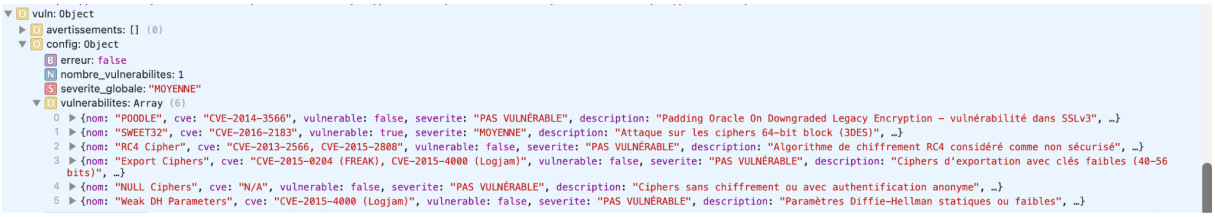
Pour tester toutes les vulnérabilités on va suivre des méthodes différentes.

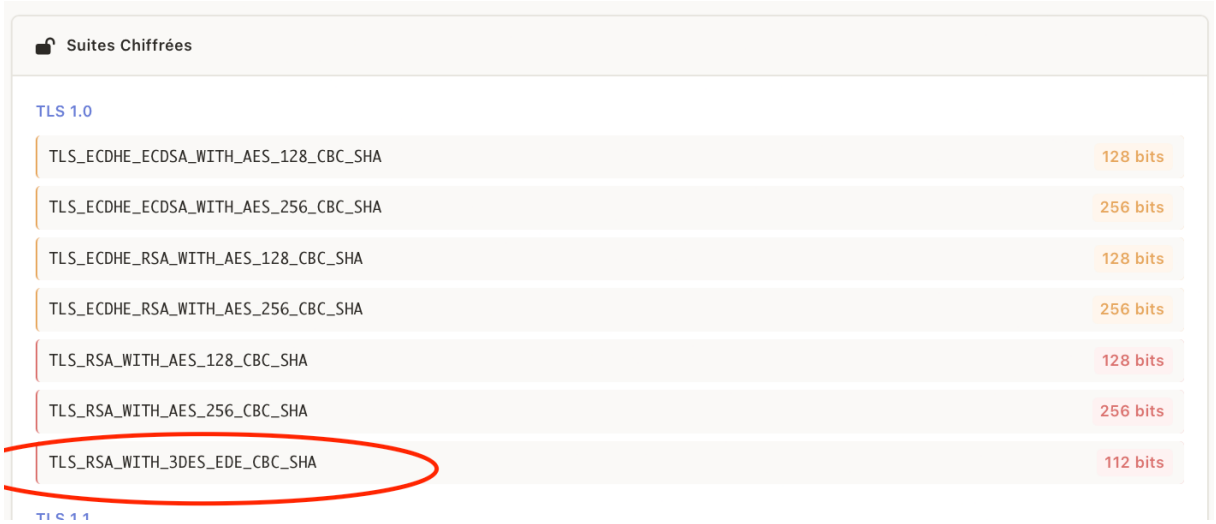
Pour les vulnérabilités de type configuration forcée : Les fonctions vérifient simplement si des protocoles ou cipher suites spécifiques sont acceptées ou pas. Pour vérifier davantage, on peut comparer nos résultats avec ceux de SSL Labs.

7.1 Test sur www.google.com

Dans le cas de google, le site est vulnérable à Sweet32 parce qu'il accepte une cipher suite 3DES.


Vulnérabilités
Heartbleed CVE-2014-0160
Fuite mémoire via l'extension Heartbeat de OpenSSL
Recommandation: Serveur non vulnérable à Heartbleed
NON VULNÉRABLE
HSTS
Max-Age: null | SubDomains: Oui | Preload: Oui
Recommandations:
• Domaine présent dans la HSTS Preload List des navigateurs.
• HTTPS est forcé par les navigateurs sans avoir besoin du header HTTP.
ACTIVÉ
POODLE CVE-2014-3566
Padding Oracle On Downgraded Legacy Encryption - vulnérabilité dans SSLv3
Détails: SSLv3 non supporté
Recommandation: SSLv3 désactivé
PAS VULNÉRABLE
SWEET32 CVE-2016-2183
Attaque sur les ciphers 64-bit block (3DES)
Détails: 3 cipher(s) 3DES trouvé(s)
Recommandation: Désactiver tous les ciphers 3DES
Ciphers affectés:
TLS_RSA_WITH_3DES_EDE_CBC_SHA
MOYENNE
RC4 Cipher CVE-2013-2566, CVE-2015-2808
Algorithme de chiffrement RC4 considéré comme non sécurisé
Détails: Aucun cipher RC4
Recommandation: RC4 non utilisé
PAS VULNÉRABLE
Export Ciphers CVE-2015-0204 (FREAK), CVE-2015-4000 (Logjam)
Ciphers d'exportation avec clés faibles (40-56 bits)
Détails: Aucun export cipher
Recommandation: Export ciphers désactivés
PAS VULNÉRABLE
NULL Ciphers
Ciphers sans chiffrement ou avec authentification anonyme
Détails: Aucun NULL cipher
Recommandation: NULL ciphers désactivés
PAS VULNÉRABLE
Weak DH Parameters CVE-2015-4000 (Logjam)
Paramètres Diffie-Hellman statiques ou faibles
Détails: Aucun DH statique
Recommandation: Configuration DH correcte
PAS VULNÉRABLE


vuln: Object
avertissements: [] (0)
config: Object
erreur: false
nombre_vulnerabilites: 1
severite_globale: "MOYENNE"
vulnerabilites: Array (6)
0 {nom: "POODLE", cve: "CVE-2014-3566", vulnerable: false, severite: "PAS VULNÉRABLE", description: "Padding Oracle On Downgraded Legacy Encryption - vulnérabilité dans SSLv3", …}
1 {nom: "SWEET32", cve: "CVE-2016-2183", vulnerable: true, severite: "MOYENNE", description: "Attaque sur les ciphers 64-bit block (3DES)", …}
2 {nom: "RC4 Cipher", cve: "CVE-2013-2566, CVE-2015-2808", vulnerable: false, severite: "PAS VULNÉRABLE", description: "Algorithme de chiffrement RC4 considéré comme non sécurisé", …}
3 {nom: "Export Ciphers", cve: "CVE-2015-0204 (FREAK), CVE-2015-4000 (Logjam)", vulnerable: false, severite: "PAS VULNÉRABLE", description: "Ciphers d'exportation avec clés faibles (40-56 bits)", …}
4 {nom: "NULL Ciphers", cve: "N/A", vulnerable: false, severite: "PAS VULNÉRABLE", description: "Ciphers sans chiffrement ou avec authentification anonyme", …}
5 {nom: "Weak DH Parameters", cve: "CVE-2015-4000 (Logjam)", vulnerable: false, severite: "PAS VULNÉRABLE", description: "Paramètres Diffie-Hellman statiques ou faibles", …}



Pour Heartbleed, on a créé un conteneur Docker vulnérable à cette vulnérabilité.

sslchecker > heartbleed-test > Dockerfile > FROM

```dockerfile
FROM ubuntu:16.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    build-essential \
    wget \
    perl \
    && rm -rf /var/lib/apt/lists/*

# Compiler OpenSSL 1.0.1f (vulnérable à Heartbleed)
RUN wget https://www.openssl.org/source/old/1.0.1/openssl-1.0.1f.tar.gz && \
    tar xzf openssl-1.0.1f.tar.gz && \
    cd openssl-1.0.1f && \
    ./config --prefix=/opt/openssl-vuln shared && \
    make && make install_sw

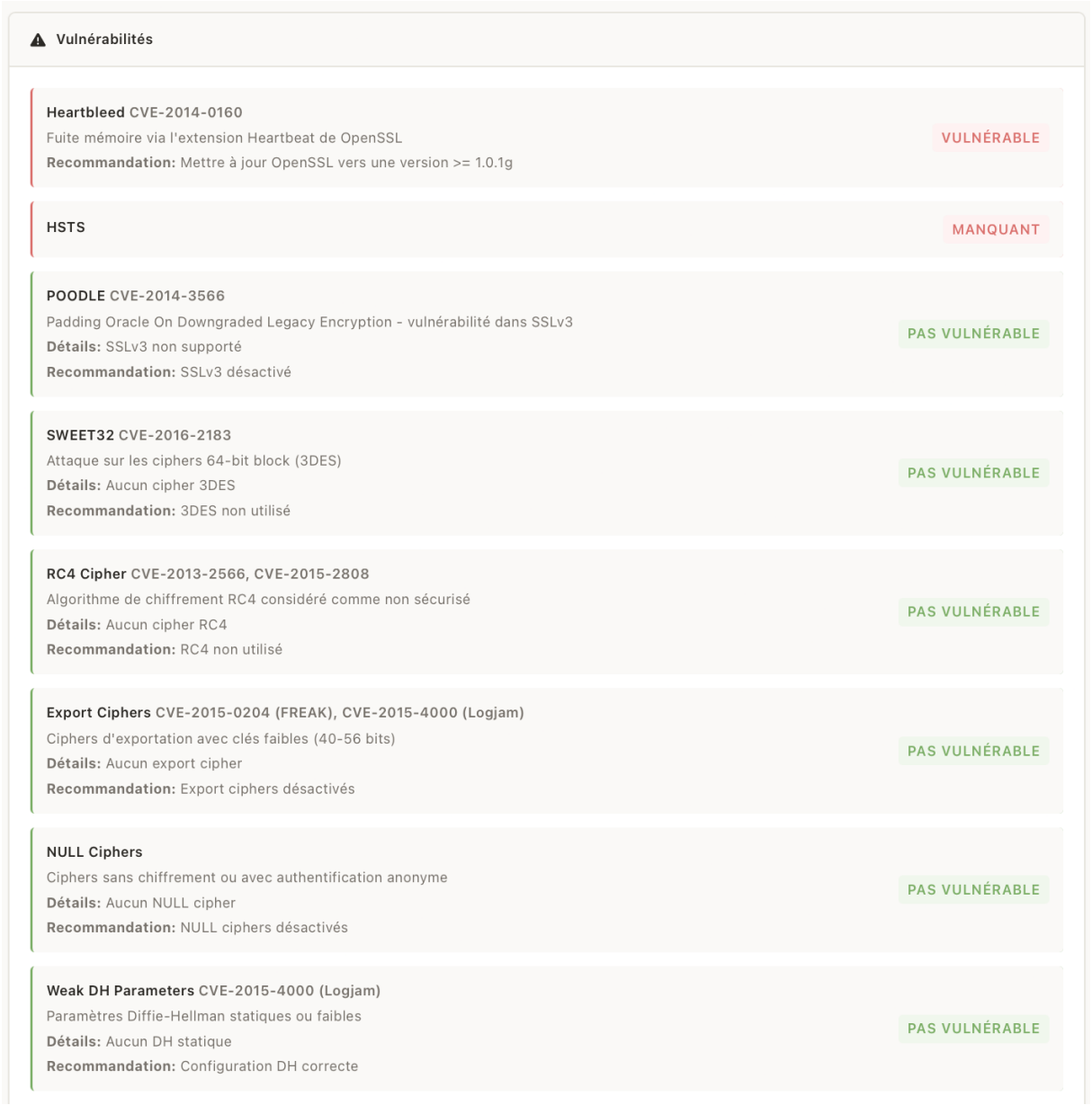
# Générer un certificat auto-signé
RUN /opt/openssl-vuln/bin/openssl req -x509 -newkey rsa:2048 \
    -keyout /key.pem -out /cert.pem -days 3650 -nodes \
    -subj "/CN=heartbleed-test"

EXPOSE 443

CMD ["/opt/openssl-vuln/bin/openssl", "s_server", \
     "-key", "/key.pem", "-cert", "/cert.pem", \
     "-accept", "443", "-www"]

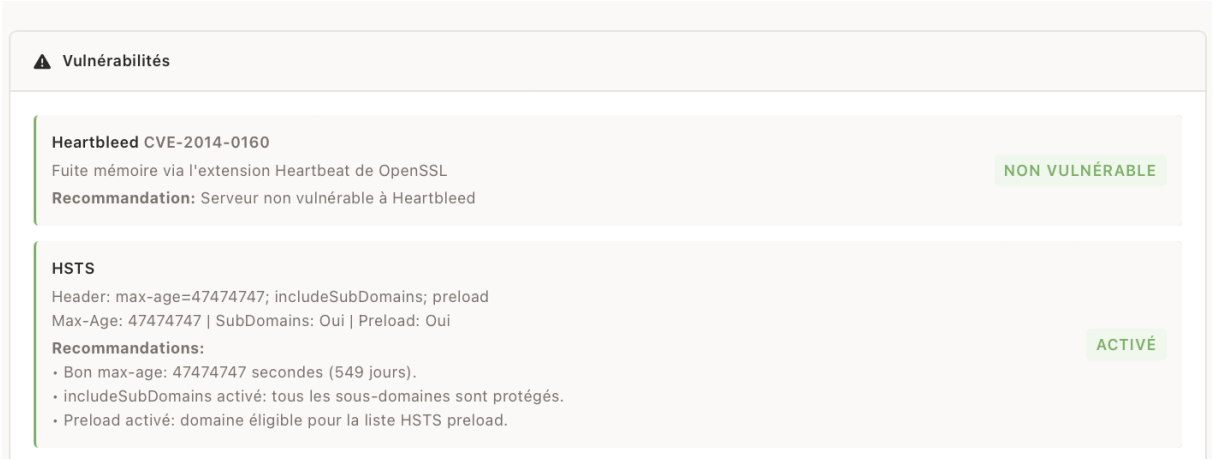
```

7.2 Test sur : host.docker.internal Port : 4433

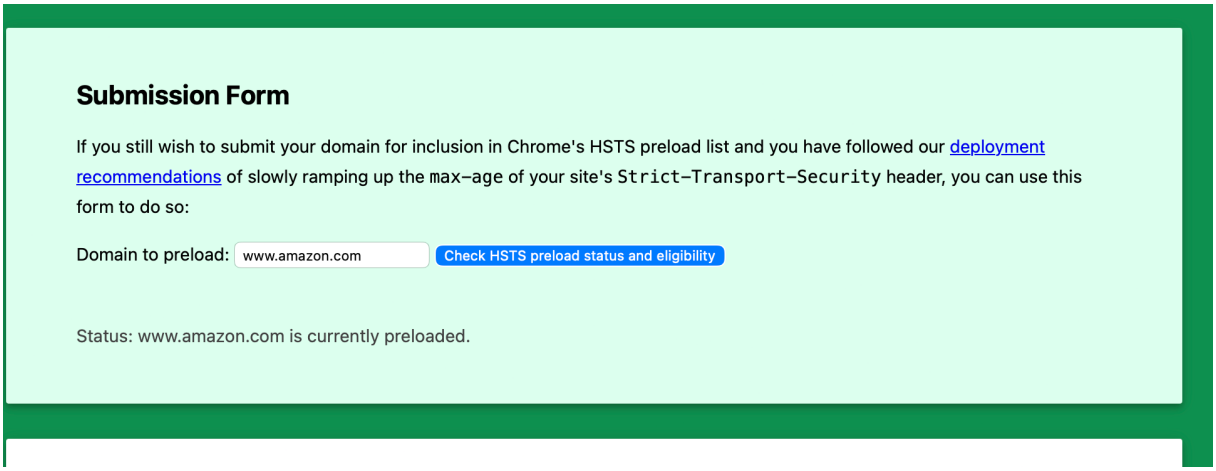


Pour HSTS : Les fonctions vérifient simplement si l'en-tête de sécurité Strict-Transport-Security est renvoyé par le serveur et si ses paramètres sont suffisants. Pour vérifier davantage, on peut comparer nos résultats en soumettant le domaine au registre officiel de Chromium (*hstspreload.org*) ou à SSL Labs

7.3 Test sur www.youtube.com



7.4 Vérification sur hstspreload.org

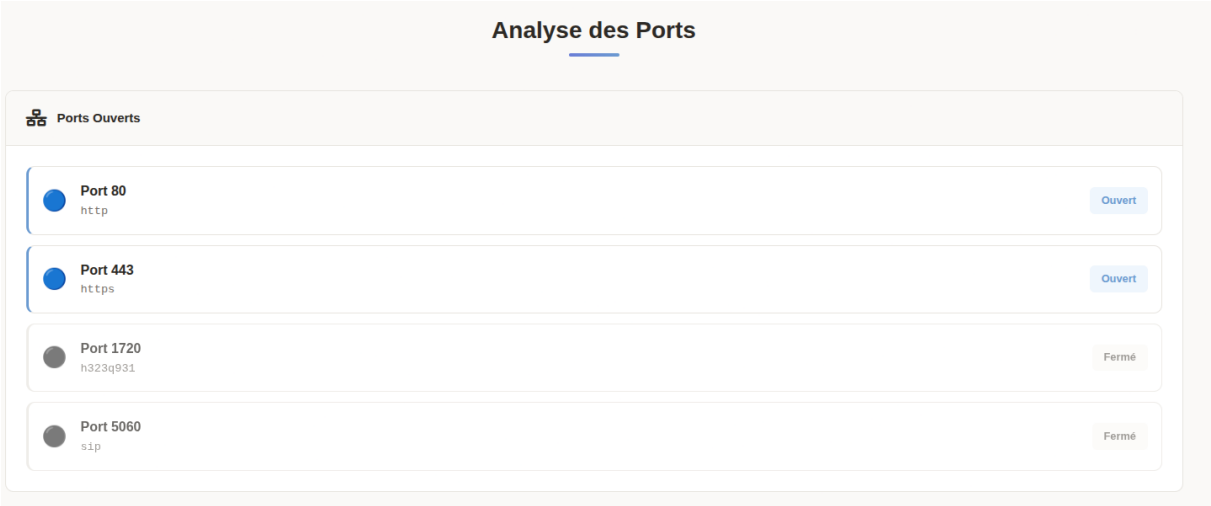


8 Module Scan de Ports

Ce module est simplement un wrapper sur la vraie commande Nmap. On a deux types de scans : le scan rapide, qui teste les 100 ports les plus communs, et le mode complet, qui lui scanne l'ensemble des ports. La réponse du module dépend entièrement de Nmap et du contexte d'exécution (utilisateur non privilégié sur Docker)

8.1 Test rapide sur sogo.univ-rouen.fr

On voit que le résultat est le même entre SSL Checker et Nmap sur la CLI

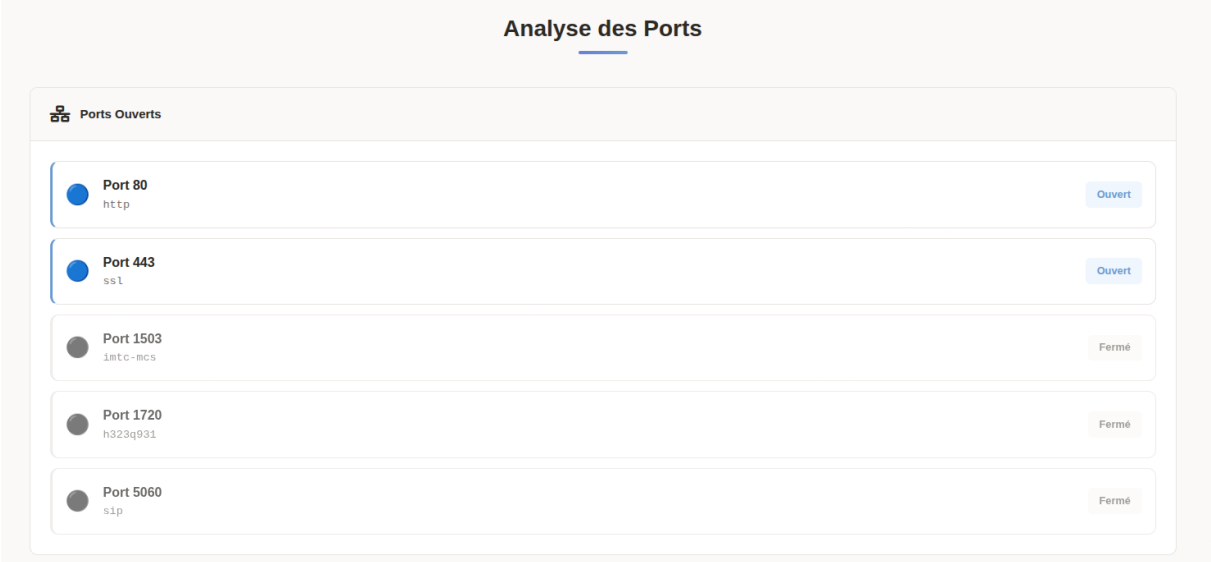



```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-14 21:53 CEST
Nmap scan report for sogo.univ-rouen.fr (10.0.128.64)
Host is up (0.013s latency).
rDNS record for 10.0.128.64: soize.univ-rouen.fr
Not shown: 96 filtered tcp ports (no-response)
PORT     STATE  SERVICE
80/tcp   open   http
443/tcp  open   https
1720/tcp closed h323q931
5060/tcp closed sip

Nmap done: 1 IP address (1 host up) scanned in 1.93 seconds
```

8.2 Test complet sur mdw.univ-rouen.fr

De même les résultats sont équivalents



```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-14 21:56 CEST
Nmap scan report for mdw.univ-rouen.fr (10.0.128.45)
Host is up (0.013s latency).
rDNS record for 10.0.128.45: mondragon.univ-rouen.fr
Not shown: 65530 filtered tcp ports (no-response)
PORT     STATE  SERVICE
80/tcp   open   http
443/tcp  open   https
1503/tcp closed imtc-mcs
1720/tcp closed h323q931
5060/tcp closed sip

Nmap done: 1 IP address (1 host up) scanned in 104.43 seconds
```